

Contents

1. Introduction.....	4
1.1 Early Automation (Pre-DevOps Era):	4
1.2 Configuration Management Tools:	4
1.3 The Rise of Ansible:	5
1.4 Modern Automation Landscape:	5
1.5 Why Choose Ansible?	5
2. Setting Up Ansible Infrastructure	6
2.1 Infrastructure Overview:	6
2.2 Launching Target Machines (Web & DB Servers):.....	6
2.3 Installing Ansible on the Control Machine:	6
3. Initiate Passwordless Authentication from control node to target nodes	11
3.1 What is Passwordless Authentication?	11
3.2 Why is it Needed for Ansible?	11
3.3 How to Initiate Passwordless Authentication	11
4. Create Ansible Inventory.....	14
4.1 Inventory	14
4.1.1 Static Inventory	14
4.1.2 Dynamic Inventory	15
4.2 Final Test with Ansible:.....	15
5. Understanding Data Structures: JSON and YAML for Ansible	16
5.1 JSON (JavaScript Object Notation)	16
5.2 YAML (YAML Ain't Markup Language)	17
5.3 Ansible Output in JSON Format	18
6. Ansible Ad-Hoc Commands.....	19
6.1 What are Ad-Hoc Commands?	20
6.2 Common Ad-Hoc Command Examples & Concepts:.....	20
6.3 Project1 – Install and run tomcat web server in web01 server from control machine using ansible ad hoc commands	21
7. Idempotency: The Core of Configuration Management	23
8. Ansible Playbooks: Structure, Execution, and Documentation	23
8.1 What are Ansible Playbooks?	23
8.1.1 Definition	23

8.1.2 Structure	24
8.1.3 Indentation is CRUCIAL.....	25
8.1.4 Example Structure	25
8.2 Executing a Playbook	27
8.3 Playbook Execution Flow & Idempotency Review	27
8.4 Debugging Playbooks (Verbosity Levels).....	27
9. Ansible Documentation	28
10. Projects	30
10.1 Project2 – Install Tomcat service in centos target machine and then copy customized index page from control node to target node.....	30
10.2 Project3 – This project installs mysql database in a target centos machine , starts the database service, creates a database and a user for the database	32
10.2.1 Error 1: Missing Python Dependencies	35
10.2.2 Error 2: Socket File Location / Access Denied	36
11. Ansible configuration	38
11.1 What is ansible configuration?	38
11.2 Why Configure Ansible?	38
11.3 Ansible Configuration File Precedence (Order of Priority)	39
11.4 Exploring the Global ansible.cfg (/etc/ansible/ansible.cfg)	40
11.5 Creating a Project-Specific ansible.cfg	41
11.6 Conclusion	42
12. Ansible Variables.....	43
12.1 Playbook Custom Variables	43
12.2 Registered variable.....	45
12.3 variables defined outside the playbook	45
12.4 Command line variable	46
12.5 Variable Precedence.....	46
12.6 Ansible Fact Variables.....	46
13. Conditions & loops.....	48
13.1 Condition	48
13.2 loops.....	49
14. Copy, file module & template module	50
14.1 Copy module.....	50
14.2 File module	50

14.3 Template Module.....	51
15. Handlers	53
15.1 Definition.....	53
15.2 Problem solved by handler.....	53
15.3 Example	53
15.3.1 How Handlers work.....	54
15.3.2 Common Use Case	55
16. Role	55
16.1 Why Use Ansible Roles?	55
16.2 What Is an Ansible Role?.....	56
16.3 Role Directory Structure.....	56
16.4 How to Convert a Playbook into a Role.....	56
16.5 Benefits of Roles	58
16.6 Ansible Galaxy Roles	58
17. Ansible for AWS Cloud Automation	59
17.1 Prerequisites: Authentication and Environment Setup.....	59
17.2 The Ansible Playbook Structure	59
17.3 Key AWS Modules and Examples	59
17.3.1 Creating an EC2 Key Pair	59
17.3.2 Launching an EC2 Instance	61
17.4 Key Takeaways	62

1. Introduction

1.1 Early Automation (Pre-DevOps Era):

- **Operating System Automation:**
 - Bash scripting for Linux task automation.
 - Batch scripting for Windows.
- **Growing Need for Diverse Automation:** Beyond OS tasks, automation became necessary for databases, virtualization, and nascent cloud computing.
- **Adoption of Programming Languages:** System administrators and operations teams began using programming languages for automation:
 - **Perl:** Popular but complex syntax.
 - **Python:** Gained significant popularity due to its vast libraries for various automation needs.
 - **Ruby:** Also used.
 - **PowerShell:** Windows-specific, capable of automating other services like VMware.

1.2 Configuration Management Tools:

- **Puppet:**
 - Pioneered the term "configuration management."
 - Initially focused on managing configurations across many servers (e.g., web server clusters) to ensure consistency and prevent "drift" (servers going out of sync).
 - Used a centralized Puppet server that applied configurations like a "law" to agent-installed target machines.
 - Later adopted as an automation tool with the rise of DevOps.
 - Built on Ruby but uses its own Domain Specific Language (DSL).
- **SaltStack:**
 - Known for its simplicity in executing commands on remote machines.
 - Often used in combination with Puppet (e.g., initial setup with SaltStack, ongoing configuration with Puppet).
 - Built on Python.
- **Chef:**
 - Offered more manageability and power, allowing users to write Ruby code.
 - Included advanced templating, features, and a good graphical user interface (GUI) for reporting.

- More complex due to multiple "moving parts" (server, client, workstation).

1.3 The Rise of Ansible:

- Developed by Michael DeHaan and later acquired by Red Hat.
- Written in Python.
- **Key Philosophy: Simplicity.** Designed for managing large infrastructures with very simple code.
- Initially focused on Linux, now supports Windows, cloud automation, networking, and databases due to extensive integrations.
- **Compared to other tools, Ansible is:**
 - **Agentless:** Unlike Puppet, Chef, or SaltStack, Ansible does not require agents on target machines. It uses existing communication protocols (SSH for Linux, WinRM for Windows, APIs for cloud/databases).
 - **Simple Setup:** A Python library, easily installed via pip or package managers.
 - **No Complex Infrastructure:** No servers, databases, or complicated storage. Configurations are in simple YAML, INI, or text formats. Output is in JSON.
 - **Human-Readable Playbooks:** Automation scripts (playbooks) are written in YAML, making them easy to read, write, and even use as documentation.
 - **Modular:** Features hundreds of modules for various tasks (e.g., package installation, service management, cloud instance launching). Can also execute shell/PowerShell commands and existing scripts.
 - **Flexible Execution:** Can perform remote execution (sending Python scripts to target machines) and local execution (for API-based tasks on the control machine).
 - **Control Machine, Not Server:** Ansible doesn't run a persistent service; it acts as a "control machine" that executes scripts and connects to targets.

1.4 Modern Automation Landscape:

- **Ansible:** Versatile for system, cloud, networking, and database automation. Ideal for change management, provisioning from scratch, and large-scale orchestration (integrating with tools like Jenkins).
- **Terraform/CloudFormation:** Primarily focused on cloud-specific automation, though Ansible can also perform cloud tasks. Terraform is often preferred for deep cloud infrastructure management.

1.5 Why Choose Ansible?

- Simplicity and ease of use.
- Agentless architecture.
- Clean and readable YAML playbooks.

- Powerful and versatile, capable of replacing many other automation tools.
- Highly integrable with other systems.

2. Setting Up Ansible Infrastructure

This session details the setup of an Ansible infrastructure, involving one control machine and multiple target servers on AWS EC2.

2.1 Infrastructure Overview:

- **Control Machine:** my machine where Ansible will be installed or an aws ubuntu instance
- **Target Machines (Clients):**
 - Two EC2 instances (CentOS) for web servers.
 - One EC2 instance (CentOS) for a DB server.
 - Later, an additional Ubuntu EC2 instance will be added as a web server.
- Ansible will be used to configure web and DB services on target machines.

2.2 Launching Target Machines (Web & DB Servers):

- **Initial Naming:** vprofile-web00 (count set to 3 for all instances). Names will be changed individually later.
- **OS:** CentOS 9 (from AWS Marketplace, ensure it's "CentOS 9 supported by supported images" – verify username is ec2-user).
- **Instance Type:** t2.micro for all three.
- **Key Pair:** Create a new key pair named client-key.pem.
- **Security Group:** Create client-sg.
 - Allow SSH (port 22) from "My IP".
- Launch instances.
- **Rename Instances:**
 - Instance 1: web01
 - Instance 2: web02
 - Instance 3: db01

2.3 Installing Ansible on the Control Machine:

- **Login to Control Machine:**

- Get the public IP of the control instance.
- Use `ssh -i <path_to_control_key.pem> ubuntu@<control_public_ip>`.
- **SSH First-Time Connection:** When connecting for the first time, SSH will ask "Are you sure you want to continue connecting?". This is presenting the server's *fingerprint* (not SSH keys). Saying "yes" stores this fingerprint in `~/.ssh/known_hosts`. Subsequent connections to the same machine won't ask again unless the `known_hosts` entry is removed.
 - To clear `known_hosts` for demonstration: `cat /dev/null > ~/.ssh/known_hosts`
- **Install Ansible (Ubuntu):**
`sudo apt update`
Add Ansible PPA: `sudo apt-add-repository --yes --update ppa:ansible/ansible`
`sudo apt install ansible -y`
- **Verify Installation:** `ansible --version` (check Ansible version and Python interpreter being used, typically Python 3).
https://docs.ansible.com/ansible/latest/installation_guide/installation_distros.html#installing-ansible-on-ubuntu

Installing Ansible on Ubuntu

Ubuntu builds are available [in a PPA here](#).

To configure the PPA on your system and install Ansible run these commands:

```
$ sudo apt update
$ sudo apt install software-properties-common
$ sudo add-apt-repository --yes --update ppa:ansible/ansible
$ sudo apt install ansible
```

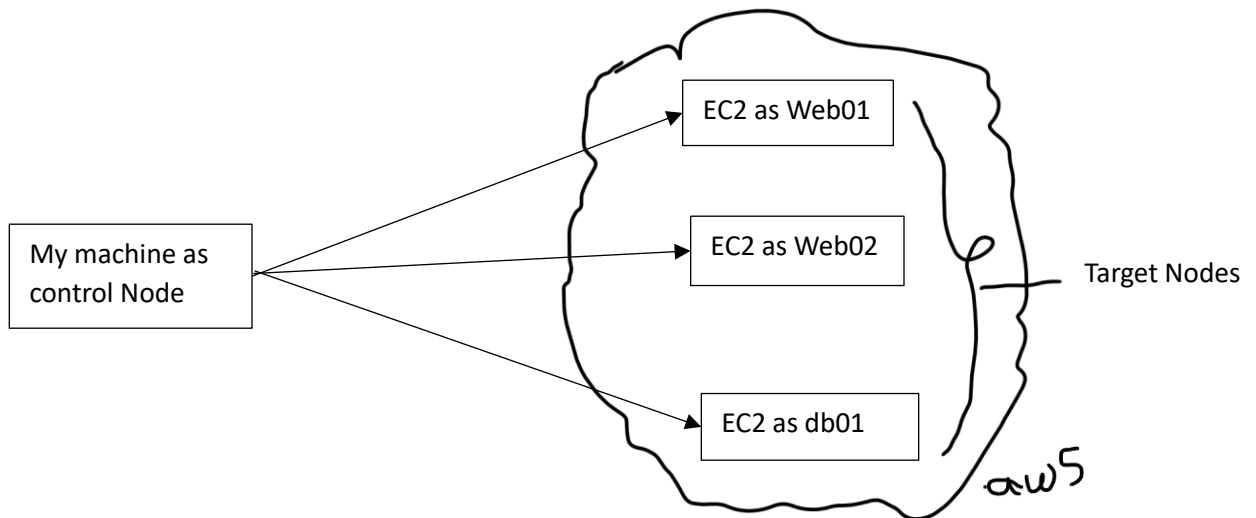
Important Considerations:

- Ansible is only installed on the **control machine**.
- Target machines (web/DB servers) do **not** need Ansible or any agents installed; Ansible uses existing SSH (for Linux) or WinRM (for Windows) to connect.
- Ansible translates playbooks into Python scripts that run on the **target machines**.

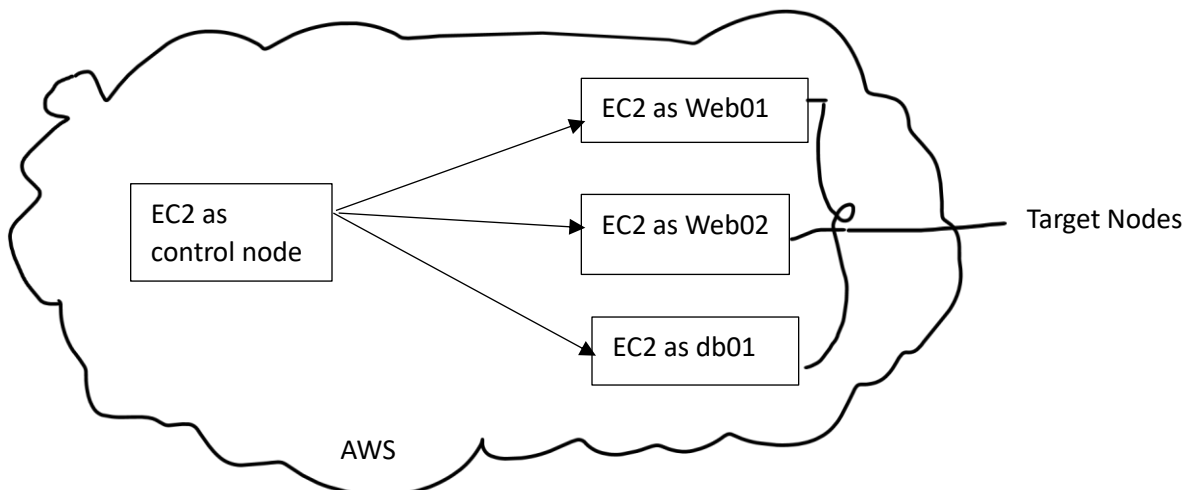
- The inventory file, which lists target machine information, will be discussed in the next session.

Lets consider some possible architectures for Ansible configuration

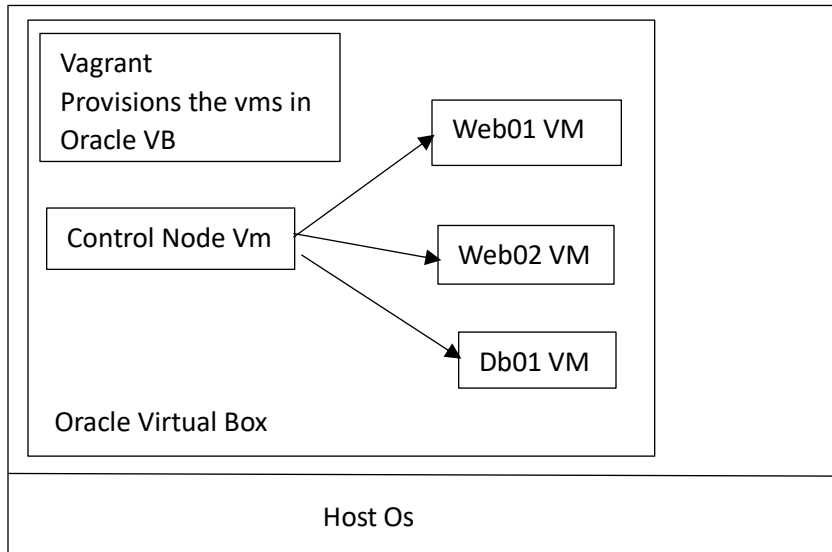
1. **Architecture1** – where control node my own machine and target nodes are ec2 instances in aws



2. **Architecture2** – where control node and target nodes are all ec2 instances in cloud. Ansible will be installed in control node.



3. **Architecture3** – Where all the control node and target nodes are in virtual machines in my laptop. They would be configured using vagrant and oracle virtual box. I would be following this architecture.



My Machine

We would be creating a folder in our machine and inside that we would create a multi vm Vagrantfile for this architecture. Then if I run vagrant up, all the servers would be up. And in the provisioner block of control node, mention ansible install commands.

```
sayan@Sayanti MINGW64 ~/vagrant-vms/ansible-vms
$ cat Vagrantfile
Vagrant.configure("2") do |config|

  # Define the Ansible Control Machine
  config.vm.define "control" do |control|
    control.vm.box = "ubuntu/focal64" # Changed to a more recent Ubuntu LTS for control
    control.vm.network "private_network", ip: "192.168.10.12"
    control.vm.hostname = "control"
    control.vm.provider "virtualbox" do |vb|
      vb.memory = "1024"
      vb.cpus = "1" # Optional: Allocate 1 CPU
    end
  end

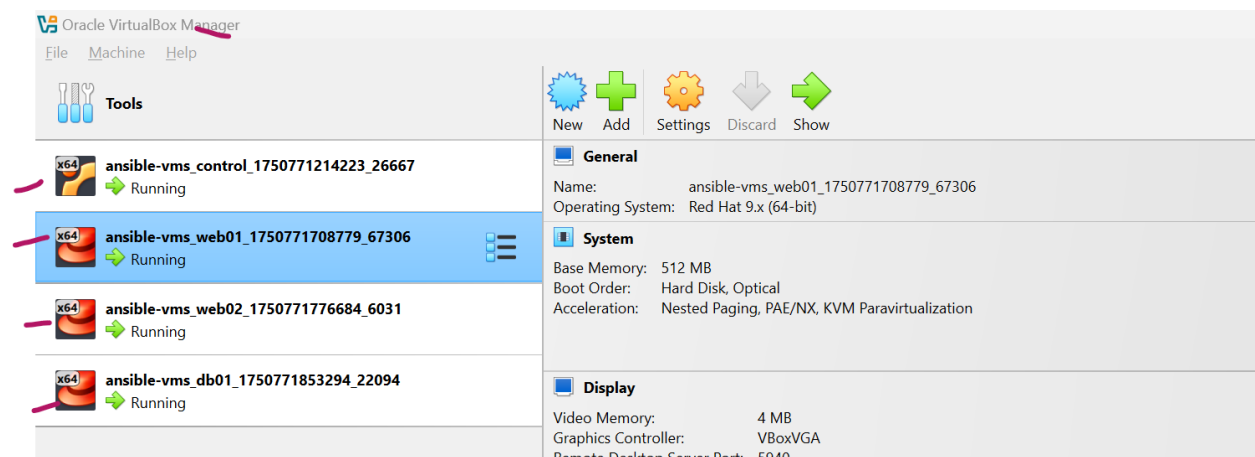
  # Provision Ansible on the control machine
  control.vm.provision "shell", inline: <<-SHELL
    sudo apt update
    sudo apt install -y software-properties-common
    sudo apt-add-repository --yes --update ppa:ansible/ansible
    sudo apt install -y ansible
    echo "Ansible installation complete on control machine."
  SHELL
end
```

```
# Define Web Server 01 (CentOS)
config.vm.define "web01" do |web01|
  web01.vm.box = "eurolinux-vagrant/centos-stream-9"
  web01.vm.network "private_network", ip: "192.168.10.13"
  web01.vm.hostname = "web01"
  web01.vm.provider "virtualbox" do |vb|
    vb.memory = "512" # Reduced memory for target VMs
    vb.cpus = "1"
  end
end

# Define Web Server 02 (CentOS)
config.vm.define "web02" do |web02|
  web02.vm.box = "eurolinux-vagrant/centos-stream-9"
  web02.vm.network "private_network", ip: "192.168.10.14"
  web02.vm.hostname = "web02"
  web02.vm.provider "virtualbox" do |vb|
    vb.memory = "512"
    vb.cpus = "1"
  end
end

# Define DB Server 01 (CentOS) - Added as per the setup description
config.vm.define "db01" do |db01|
  db01.vm.box = "eurolinux-vagrant/centos-stream-9"
  db01.vm.network "private_network", ip: "192.168.10.15" # Changed IP to avoid conflict with web03
  db01.vm.hostname = "db01"
  db01.vm.provider "virtualbox" do |vb|
    vb.memory = "512"
    vb.cpus = "1"
  end
end
```

Once the file run successfully we can see all the nodes in Oracle virtualbox



Run → `vagrant ssh control`

Then check the ansible version installed in the control node

```
vagrant@control:~$ ansible --version
ansible [core 2.12.10]
  config file = /etc/ansible/ansible.cfg
  configured module search path = ['/home/vagrant/.ansible/plugins/modules', '/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/lib/python3/dist-packages/ansible
  ansible collection location = /home/vagrant/.ansible/collections:/usr/share/ansible/collections
  executable location = /usr/bin/ansible
  python version = 3.8.10 (default, Mar 18 2025, 20:04:55) [GCC 9.4.0]
  jinja version = 2.10.1
  libyaml = True
```

3. Initiate Passwordless Authentication from control node to target nodes

3.1 What is Passwordless Authentication?

Passwordless authentication, refers to a method of logging into a remote server without needing to type a password. Instead, it relies on SSH key pairs – a public key and a private key.

Public Key: This key is placed on the target server. It's like a digital lock.

Private Key: This key is kept securely on the client (control node in Ansible's case). It's like the digital key that matches the lock.

When a connection attempt is made, the client proves its identity by demonstrating that it possesses the private key corresponding to the public key on the server, without ever transmitting the private key itself.

3.2 Why is it Needed for Ansible?

Passwordless authentication is crucial for Ansible deployments for several reasons:

Automation: Ansible is designed for automation. Requiring a password for every SSH connection to potentially hundreds or thousands of servers would defeat the purpose of automation and make large-scale deployments impractical.

Idempotency & Reliability: Ensures that Ansible can consistently connect and execute tasks without manual intervention, which is essential for idempotent operations.

3.3 How to Initiate Passwordless Authentication

Case1: Lets understand this for vagrant servers

As shown in the architecture before, 1st I am doing vagrant ssh control. Once I am inside control node will try to ssh into another node. See, it would be asking for password.

```
vagrant@control:~$ ssh vagrant@192.168.10.18
The authenticity of host '192.168.10.18 (192.168.10.18)' can't be established.
ECDSA key fingerprint is SHA256:Nd7x0kuJNrR8iMMKVmYvQ3oZUpq0APLjslND02wXeq.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.10.18' (ECDSA) to the list of known hosts.
vagrant@192.168.10.18's password: |
```

But we want passwordless less from control node. How can we do that?

The process involves generating an SSH key pair on the control node and then securely copying the public key to all target nodes.

Steps:

Step 1: Generate SSH Key Pair on the Control Node

Log in to your Ansible control node. (e.g., `vagrant ssh control` if using the Vagrant setup)

Generate the key pair:

```
ssh-keygen -t rsa -b 4096 -C "ansible_key"
```

-t rsa: Specifies the RSA algorithm.

-b 4096: Specifies a 4096-bit key length (stronger than the default 2048-bit).

-C "ansible_key": Adds a comment to the public key (optional, but good for identification).

When prompted for a file to save the key, press Enter to accept the default location (`~/.ssh/id_rsa`).

When prompted for a passphrase, it's highly recommended to press Enter twice to leave it empty for automation purposes. This command will create two files in the `~/.ssh/` directory:

`id_rsa` (your private key) - KEEP THIS FILE SECURE AND NEVER SHARE IT!

`id_rsa.pub` (your public key) - This is the key you will share.

```
vagrant@control:~$ ls ~/.ssh
authorized_keys  id_rsa  id_rsa.pub  known_hosts
```

Step 2: Copy the Public Key to Target Nodes [Run the command in control node only]

There are a few ways to do this:

a) Using `ssh-copy-id` (Recommended if you have initial password access):

This is the easiest and most secure method if you can initially log in to the target nodes using a password.

From your control node, run:

```
ssh-copy-id -i ~/.ssh/id_rsa.pub <username>@<target_ip_address>
```

-i `~/.ssh/id_rsa.pub`: Specifies the public key to use.

<username>: The username on the target node (e.g., `vagrant` for Vagrant VMs, `ec2-user` for CentOS on EC2, `ubuntu` for Ubuntu on EC2).

<target_ip_address>: The IP address of your target node (e.g., `192.168.10.13` for `web01`).

You will be prompted for the password of the <username> on the target node. Enter it (First Time only – `vagrant` for `vagrant` servers)

`ssh-copy-id` will then securely add your public key to the `~/.ssh/authorized_keys` file on the target node.

Repeat this for `web01`, `web02`, `db01`, and `web03_ubuntu`.

```

vagrant@control:~$ ls ~/.ssh
authorized_keys  id_rsa  id_rsa.pub  known_hosts
vagrant@control:~$ ssh-copy-id -i ~/.ssh/id_rsa.pub vagrant@192.168.10.18
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/vagrant/.ssh/id_rsa.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already
installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to install
the new keys
vagrant@192.168.10.18's password:
Number of key(s) added: 1
Now try logging into the machine, with: "ssh 'vagrant@192.168.10.18'"
and check to make sure that only the key(s) you wanted were added.

```

If control node is my physical machine and target nodes are aws instances use below variation of ssh-copy-id

ssh-copy-id -f "-o IdentityFile <PATH TO PEM FILE>" ubuntu@<INSTANCE-PUBLIC-IP>

- ssh-copy-id: This is the command used to copy your public key to a remote machine.
- -f: This flag forces the copying of keys, which can be useful if you have keys already set up and want to overwrite them.
- "-o IdentityFile ": This option specifies the identity file (private key) to use for the connection. The -o flag passes this option to the underlying ssh command.
- ubuntu@: This is the username (ubuntu) and the IP address of the remote server you want to access.

Step 3: Test Passwordless Authentication in control node

Now From your control node, try to SSH to a target node without specifying a password or key:

ssh <username>@<target_ip_address>

[For Vagrant server use the private ip as target_ip_address

For EC2 instance it would be public ip]

If configured correctly, you should log in directly without being prompted for a password.

```

vagrant@control:~$ ssh vagrant@192.168.10.18
This box was generated by EuroLinux

Any suggestions are welcome at https://github.com/EuroLinux/cloud-images-rfc/

Happy using.
To delete this message use:
echo '' > /etc/motd
[vagrant@web03 ~]$

```

Now, we can ssh into web03 from control node without any password or key.

4. Create Ansible Inventory

Once passwordless SSH is set up, ensure your Ansible inventory file on the control node is configured to use the correct user and not to specify a password or a specific private key

4.1 Inventory

Ansible inventory file is a fundamental component of Ansible that defines the hosts (remote systems) that you want to manage and the groups those hosts belong to. The inventory file can be static (a simple text file) or dynamic (generated by a script). It provides Ansible with the information about the remote nodes to communicate with during its operations.

4.1.1 Static Inventory

A static inventory file is typically a plain text file (usually named hosts or inventory) and is structured in INI or YAML format. Here are examples of both formats:

INI Format

```
# inventory file: hosts

[webservers]

web1.example.com

web2.example.com

[dbservers]

db1.example.com

db2.example.com

[all:vars]

ansible_user=admin

ansible_ssh_private_key_file=/path/to/key
```

YAML

```
# inventory file: hosts.yaml

all:

  vars:

    ansible_user: admin

    ansible_ssh_private_key_file: /path/to/key

  children:
```

webservers:

hosts:

web1.example.com:

web2.example.com:

dbservers:

hosts:

db1.example.com:

db2.example.com:

4.1.2 Dynamic Inventory

A dynamic inventory is generated by a script or plugin and can be used for environments where hosts are constantly changing (e.g., cloud environments). The script or plugin fetches the list of hosts from a source like AWS, GCP, or any other dynamic source.

Here is an example of a dynamic inventory script for AWS EC2: script mentioned in below link

`#!/usr/bin/env python`

<https://github.com/iam-veeramalla/ansible-zero-to-hero/blob/main/Day-02/02-inventory.md#inventory>

4.2 Final Test with Ansible:

Ansible -i inventory <Adhoc command or Playbook.yml>

Inventory file

```
vagrant@control:~/vprofile/ansible/exercise2$ cat inventory
all:
  hosts:
    web01:
      ansible_host: 192.168.10.13
      ansible_user: vagrant

    web02:
      ansible_host: 192.168.10.14
      ansible_user: vagrant

    db01:
      ansible_host: 192.168.10.15
      ansible_user: vagrant

#...grouping.....

children:
  webservers:
    hosts:
      web01:
      web02:
  dbservers:
    hosts:
      db01:
  parent_grp:
    children:
      webservers:
      dbservers:
```

```
128 ansible -i inventory -m ping parent_grp
129 ansible -i inventory -m ping dbservers
130 ansible -i inventory -m ping webservers
```

```
vagrant@control:~/vprofile/exercise1$ ansible -i inventory -m ping web01
web01 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
```

Here we are using a module named ping to check whether ansible control node able to reach all the target nodes mentioned in inventory. If successful it would return a pong message as return.

5. Understanding Data Structures: JSON and YAML for Ansible

This session highlights the evolution from less readable formats like pom.xml to more human-friendly data structures like JSON and YAML, which are extensively used in DevOps tools like Ansible.

5.1 JSON (JavaScript Object Notation)

- **Foundation:** JSON is conceptually similar to Python dictionaries. If you understand Python dictionaries (key-value pairs, lists, nested structures), you understand the core of JSON.
- **Structure:**
 - **Keys:** Strings (usually enclosed in double quotes).
 - **Values:** Can be strings, numbers, booleans, arrays (lists), objects (dictionaries), or null.
 - Uses curly braces {} for objects (dictionaries) and square brackets [] for arrays (lists).
 - Data is typically structured vertically for readability, with proper indentation.
- **Example (Python Dictionary to JSON):**

Python

Python Dictionary (Horizontal, less readable)

```
data = {"DevOps": ["Ansible", "Jenkins"], "development": ["Python", "Java"], "ansible_facts": {"os": "Linux"}}
```


Python Dictionary (Vertical, more readable - this form is essentially JSON)

```
data = {  
    "DevOps": [  
        "Ansible",  
        "Jenkins"  
    ],  
    "development": [  
        "Python",  
        "Java"  
    ],  
    "ansible_facts": {  
        "os": "Linux"  
    }  
}
```

- **Adoption:** JSON is widely adopted in modern tools and APIs for data exchange due to its simplicity and readability compared to XML.
- **Ansible's Use:** Ansible modules return their output in **JSON format**, making it readable after execution.

5.2 YAML (YAML Ain't Markup Language)

- **Simplicity:** YAML is an even more human-readable and writable data serialization language, designed for configuration files.
- **Structure:**
 - **No opening/closing tags or braces/brackets (mostly):** Relies heavily on **indentation** (spaces) to define structure, similar to Python.
 - **Key-Value Pairs:** key: value (a space after the colon is crucial).
 - **Lists (Arrays):** Elements are indicated by a hyphen - followed by a space, on separate lines, with consistent indentation.
 - **Dictionaries (Objects):** Nested key-value pairs are created by indenting them under the parent key.
 - Double quotes for strings are optional unless the string contains special characters.

- **Example (JSON to YAML):**

YAML

DevOps:

- Ansible
- Jenkins

development:

- Python
- Java

ansible_facts:

os: Linux

- **Ansible's Core: Ansible playbooks are written in YAML format.** This design choice makes playbooks highly readable and easy to write, even for those without extensive programming backgrounds.
- **YAML Syntax Indicators (Optional):**
 - ---: Three hyphens at the beginning of a file can optionally indicate the start of a YAML document.

5.3 Ansible Output in JSON Format

- When you execute an Ansible module or playbook, the output is presented in JSON.
- **Common Output Keys:**
 - **changed:** A boolean (true or false) indicating whether the executed task made any changes to the target system.
 - false: For informational tasks (like ping) or if the desired state was already met (idempotency).
 - true: If the task performed an actual modification (e.g., installing a package, changing a configuration file).
 - **ping:** For the ping module, the value is typically pong.
 - **ansible_facts:** Often present, especially if facts gathering is enabled, containing gathered information about the target system.
 - Other keys will depend on the specific module and its output.

```
vagrant@control:/vagrant/Project4/ansible$ ansible -i inventory.yml -m ping docker_server2
docker_server2 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
```

In essence, Ansible uses YAML for its inputs (playbooks) and provides JSON for its outputs (results), leveraging the strengths of both human-readable and machine-parseable data formats.



```
Python Dictionary
{"DevOps": ["AWS", "Jenkins", "Python", "Ansible"], "Development": ["Java", "NodeJS", ".net"], "ansible_facts": {}}

JSON
{
  "DevOps": [
    "AWS",
    "Jenkins",
    "Python",
    "Ansible"
  ],
  "Development": [
    "Java",
    "NodeJS",
    ".net"
  ],
  "ansible_facts": {}
}
```

Once we remove the braces it becomes YAML



```
YAML
DevOps:
- AWS
- Jenkins
- Python
- Ansible

Development:
- Java
- NodeJS
- .net

ansible_facts:
  python: /usr/bin/python
  version: 2.7
```

6. Ansible Ad-Hoc Commands

This session introduces Ansible **Ad-Hoc commands**, a powerful feature for quick, single-task operations, and delves into the concept of **idempotency**, a core principle of configuration management.

6.1 What are Ad-Hoc Commands?

- **Definition:** Ad-hoc commands are one-liner Ansible commands executed directly from the command line, without needing to write a formal playbook.
- **Purpose:** They are ideal for tasks that are rarely repeated, quick fixes, or for testing connectivity and module functionality.
- **Contrast with Playbooks:** While playbooks (written in YAML) are the preferred method for managing infrastructure (due to version control, reusability, and structure), ad-hoc commands offer immediate execution.
- **Basic Syntax:** `ansible <pattern> -m <module_name> -a "<module_arguments>" -i <inventory_file>`
 - `<pattern>`: Specifies the target hosts (e.g., `web01`, `webservers`, `all`).
 - `-m <module_name>`: Specifies the Ansible module to use (e.g., `ping`, `yum`, `apt`, `service`, `copy`).
 - `-a "<module_arguments>"`: Passes arguments to the module (e.g., `name=httpd state=present`).
 - `-i <inventory_file>`: Specifies the path to your inventory file.

6.2 Common Ad-Hoc Command Examples & Concepts:

- **Ping Module (`ansible.builtin.ping`):**
 - **Command:** `ansible webservers -m ansible.builtin.ping -i inventory.ini`
 - **Purpose:** Tests connectivity to target hosts.
 - **Output:** Returns `pong` on success. `changed` will be `false` as it makes no changes.
- **Package Management (`ansible.builtin.yum` / `ansible.builtin.apt`):**
 - **Example (yum for CentOS):** `ansible web01 -m ansible.builtin.yum -a "name=httpd state=present" -i inventory.ini`
 - **Common Arguments:** `name` (package name), `state` (present to install, absent to uninstall).
 - **Privilege Escalation (`--become`):** Many tasks (like package installation) require root privileges. You must add `--become` (or `-b`) to the command: `ansible webservers -m ansible.builtin.yum -a "name=httpd state=present" --become -i inventory.ini` This tells Ansible to use `sudo` on the target machine.

- **Service Management (ansible.builtin.service):**

- **Example:** `ansible webserver -m ansible.builtin.service -a "name=httpd state=started enabled=yes" --become -i inventory.ini`
- **Purpose:** Start, stop, restart, enable/disable services.
- **Common Arguments:** name (service name), state (started, stopped, restarted), enabled (yes/no to set service to start on boot).

- **File Copying (ansible.builtin.copy):**

Example: `ansible webserver -m ansible.builtin.copy -a "src=./index.html dest=/var/www/html/index.html" --become -i inventory.ini`

Purpose: Copy files from the control node to target nodes.

Common Arguments: src (source path on control node), dest (destination path on target node, including filename).

6.3 Project1 – Install and run tomcat web server in web01 server from control machine using ansible ad hoc commands

- Ensure passwordless authentication is configured for web01 in control node

Command:- **ansible -i inventory -m ping web01**

```
vagrant@control:~/vprofile/ansible/ansible/exercise2$ ansible -i inventory -m ping web01
[WARNING]: Platform linux on host web01 is using the discovered Python interpreter at /usr/bin/python3.9, but future install
change the meaning of that path. See https://docs.ansible.com/ansible-core/2.17/reference_appendices/interpreter_discovery.h
web01 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.9"
  },
  "changed": false,
  "ping": "pong"
}
```

- Install httpd in web01

Command:- **ansible -i inventory -b -m yum -a "name=httpd state=present" web01**

```
vagrant@control:~/vprofile/ansible/ansible/exercise2$ ansible -i inventory -b -m yum -a "name=httpd state=present" web01
[WARNING]: Platform linux on host web01 is using the discovered Python interpreter at /usr/bin/python3.9, but future install
change the meaning of that path. See https://docs.ansible.com/ansible-core/2.17/reference_appendices/interpreter_discovery.h
web01 | CHANGED => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.9"
  },
  "changed": true,
  "msg": "",
  "rc": 0,
  "results": [
    "installed: httpd-filesystem-2.4.62-4.el9.noarch",
  ]
}
```

- Start & Enable Httpd in Web01

Command:- **ansible -i inventory -b -m systemd -a "name=httpd state=started enabled=yes" web01**

```
vagrant@control:~/vprofile/ansible/ansible/exercise2$ ansible -i inventory -b -m systemd -a "name=httpd state=started enabled=yes" web01
[WARNING]: Platform linux on host web01 is using the discovered Python interpreter at /usr/bin/python3.9, but future installation of another Python
web01 | CHANGED => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.9"
  },
  "changed": true,
  "enabled": true,
  "name": "httpd",
  "state": "started",
}
```

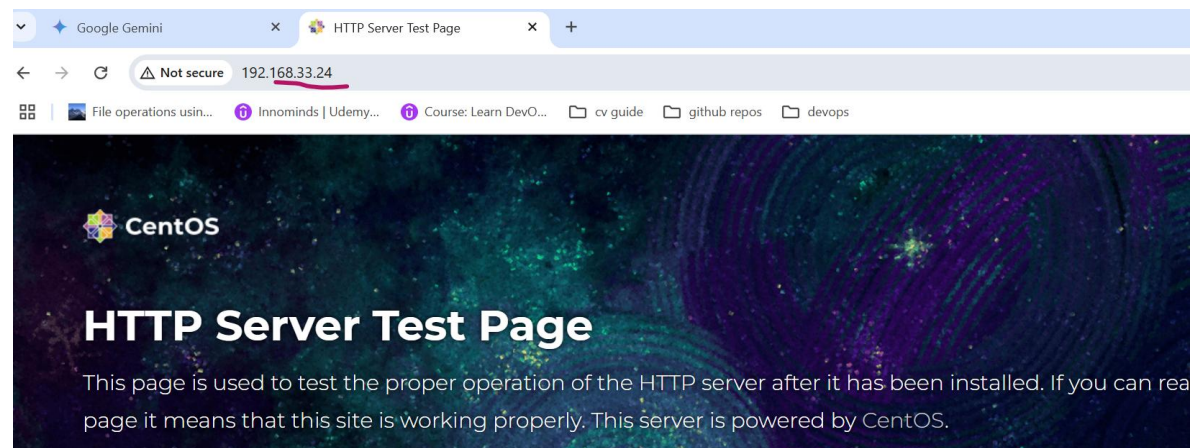
- Check the status of the service whether its running

Command:- **ansible -i inventory -b -m shell -a "systemctl status httpd" web01**

```
vagrant@control:~/vprofile/ansible/ansible/exercise2$ ansible -i inventory -b -m shell -a "systemctl status httpd" web01
[WARNING]: Platform linux on host web01 is using the discovered Python interpreter at /usr/bin/python3.9, but future installation of another Python
web01 | CHANGED | rc=0 >>
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
   Active: active (running) since Wed 2025-07-09 05:55:26 UTC; 1min 16s ago
     Docs: man:httpd.service(8)
   Main PID: 29729 (httpd)
   Status: "Total requests: 0; Idle/Busy workers 100/0; Requests/sec: 0; Bytes served/sec: 0 B/sec"
     Tasks: 177 (limit: 2681)
   Memory: 14.0M
    CPU: 157ms
```

- Check in the browser with web01 ip whether the service running or not

Command:- **http://<web01_ip>**



All commands:-

```
ansible -i inventory -m ping web01
49 ansible -i inventory -b -m yum -a "name=httpd state=present" web01
50 ansible -i inventory -b -m systemd -a "name=httpd state=started enabled=yes" web01
51 ansible -i inventory -b -m shell -a "systemctl status httpd" web01
```

Next Steps: Playbooks

- While ad-hoc commands are useful, the full power of Ansible for complex automation and infrastructure management comes from **playbooks**, which provide a structured way to define multiple tasks, roles, and conditional logic in YAML. This will be covered in the next session.

7. Idempotency: The Core of Configuration Management

- **Definition:** Idempotency means that an operation can be applied multiple times without changing the result beyond the initial application. In Ansible, this means if a task is run when the target system is already in the desired state, Ansible will report "no change" and not re-apply the action.
- **How Ansible Achieves It:** Ansible modules are designed to compare the *current state* of the target with the *desired state* specified in the task.
 - If the states are different, Ansible applies the change, and the output `changed` will be `true`.
 - If the states are identical, Ansible does nothing, and the output `changed` will be `false`.
- **Advantage over Traditional Scripting:** Unlike simple scripts that might re-execute commands every time (potentially causing errors, unnecessary resource usage, or downtime), configuration management tools like Ansible ensure changes are only applied when needed. This leads to:
 - **Reliability:** Systems remain in a consistent state.
 - **Efficiency:** Avoids redundant operations.
 - **Predictability:** Outcomes are consistent regardless of how many times a playbook is run.
 - **Example:** Running `state=present` for an already installed package will result in `changed: false` with a "nothing to do" message. Changing `index.html` on the control node and rerunning the copy task will show `changed: true` because the source and destination content differ.

8. Ansible Playbooks: Structure, Execution, and Documentation

This session introduces Ansible playbooks, which are the core of Ansible automation, detailing their structure, how to execute them, debugging techniques, and navigating Ansible documentation for modules.

8.1 What are Ansible Playbooks?

8.1.1 Definition: Playbooks are YAML-formatted files that define a set of automation tasks for Ansible to execute on target hosts. They are analogous to scripts in other languages (e.g., Bash, Python).

8.1.2 Structure: A playbook is a **list of plays**.

- **Plays:** Each play is a dictionary that targets a specific group of hosts and contains a list of tasks.
 - Starts with a hyphen (-) to denote a list item.
 - **name:** (Optional, but highly recommended) A descriptive name for the play.
 - **hosts:** Specifies the target hosts or host groups from the inventory (e.g., webservers, dbservers, all).
 - **become: yes:** (Optional) Elevates privileges (e.g., sudo) for all tasks within that play.

```
--  
- name: DBserver setup  
  hosts: dbservers  
  become: yes  
  tasks:  
    - name: Install mariadb-server  
      yum:  
        name: mariadb-server  
        state: present
```

Example of a play

- **tasks:** A list of dictionaries, where each dictionary defines a single task.
 - Task:** Each task is a dictionary:
 - ✓ Starts with a hyphen (-) to denote a list item under tasks.
 - ✓ **name:** (Optional, but highly recommended) A descriptive name for the task. This makes playbooks much more readable and serves as self-documentation.
 - ✓ **Module Call:** The module name (e.g., ansible.builtin.yum, ansible.builtin.service, ansible.builtin.copy,) is a key, and its value is a dictionary of arguments/options for that module.
 - Module parameters


```

- name: Start MariaDB service
  ansible.builtin.service:
    name: mariadb
    state: started
    enabled: yes

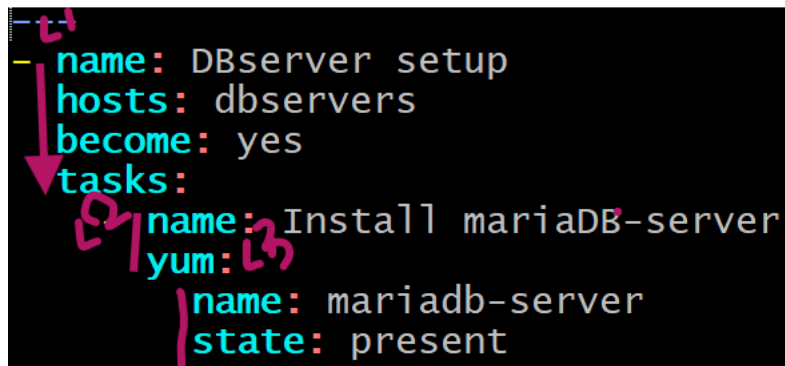
- name: Install PyMySQL dependency
  ansible.builtin.yum:
    name: python3-PyMySQL # Or other specific package name
    state: present

```

Example of Tasks

8.1.3 Indentation is CRUCIAL: YAML relies on consistent indentation (typically 2 spaces per level).

- Play (Level 1)
- Play details (name, hosts, become) (Level 1)
- Tasks (Level 1)
- Each task item (- name:) (Level 2)
- Module name (Level 2)
- Module arguments (Level 3)



```

- name: DBserver setup
  hosts: dbservers
  become: yes
  tasks:
    - name: Install mariaDB-server
      yum:
        name: mariadb-server
        state: present

```

8.1.4 Example Structure:

YAML

--- # Optional: YAML document start indicator

- name: Webserver Setup Play

hosts: webservers

become: yes

tasks:

- name: Install HTTPD package

ansible.builtin.yum:

name: httpd

state: present

- name: Start HTTPD service

ansible.builtin.service:

name: httpd

state: started

enabled: yes

- name: DBserver Setup Play

hosts: dbservers

become: yes

tasks:

- name: Install MariaDB Server package

ansible.builtin.yum:

name: mariadb-server

state: present

- name: Start MariaDB service

ansible.builtin.service:

name: mariadb

state: started

enabled: yes

8.2 Executing a Playbook

- **Command:** `ansible-playbook -i <inventory_path> <playbook_name.yml>`
 - Example: `ansible-playbook -i inventory.ini web-db.yml`

8.3 Playbook Execution Flow & Idempotency Review

- When a playbook runs, Ansible processes each play and its tasks sequentially on the targeted hosts.
- **Gathering Facts:** By default, the first task in every play is "Gathering Facts". Ansible runs the setup module to collect extensive information about the target machine (OS, IP addresses, memory, etc.) and stores it in runtime variables (`ansible_facts`). This does not involve a "changed" state.
- **Idempotency in Playbooks:**
 - Ansible modules are designed to be idempotent.
 - If a task tries to apply a configuration that the target system already has, Ansible will report `changed: false` (or `ok`) and skip the actual operation.
 - If the target system is *not* in the desired state, Ansible will apply the change, and the output will show `changed: true`.
 - This ensures that running a playbook multiple times safely brings the system to the desired state without unnecessary re-applications.

```
vagrant@control:~/vprofile/ansible/ansible/exercise6$ ansible-playbook -i inventory db.yml
PLAY [DBserver setup] *****
TASK [Gathering Facts] *****
ok: [db01]
TASK [Install mariaDB-server] *****
ok: [db01]
TASK [Start MariaDB service] *****
ok: [db01]
TASK [Install PyMySQL dependency] *****
ok: [db01]
TASK [Create a new database with name 'bobdata'] *****
changed: [db01]
```

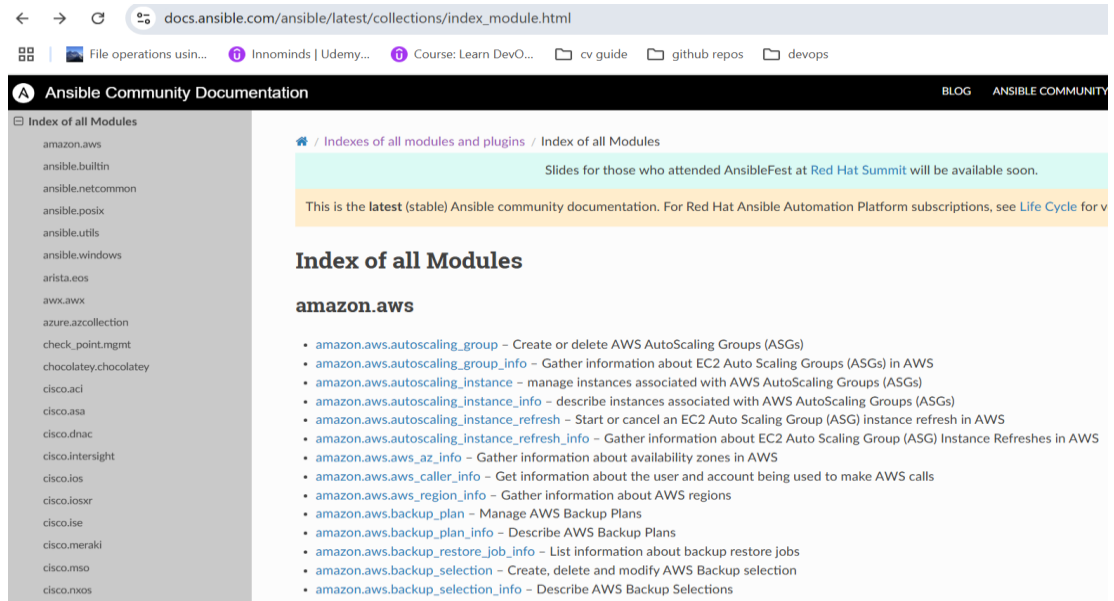
8.4 Debugging Playbooks (Verbosity Levels)

- You can increase the verbosity of the playbook output for debugging:
 - `-v`: (1 'v') Shows more detailed output, including JSON data for module results.

- -vv: (2 'v's) Even more detail, including playbook path, line numbers, and some internal Ansible workings.
- -vvv: (3 'v's) Very verbose, showing SSH commands, identity files, and user connections.
- -vvvv: (4 'v's) The highest level of verbosity, providing extremely detailed output.
- **Syntax Check (--syntax-check):**
 - `ansible-playbook --syntax-check <playbook_name.yml>`
 - Checks the YAML syntax of the playbook without running any tasks. Essential for catching common YAML formatting errors.
 - **Caveat:** While helpful, the reported line number for a syntax error might sometimes be misleading due to YAML's reliance on indentation. Always review the surrounding lines.
- **Dry Run (--check or -C):**
 - `ansible-playbook --check -i <inventory_path> <playbook_name.yml>`
 - Performs a "dry run" or "what-if" execution. Ansible simulates applying changes without actually modifying the target systems.
 - It indicates what *would* change if the playbook were run for real.
 - **Important:** Dry runs are a good practice but **not a guarantee**. Some tasks involving runtime variables or complex logic might behave differently in a dry run versus actual execution.

9. Ansible Documentation

- **Primary Resource:** docs.ansible.com/ansible/latest/
- **Navigating Modules:**
 - Go to "Using Ansible modules and plugins" or directly to "Index of all modules".
https://docs.ansible.com/ansible/latest/collections/index_module.html

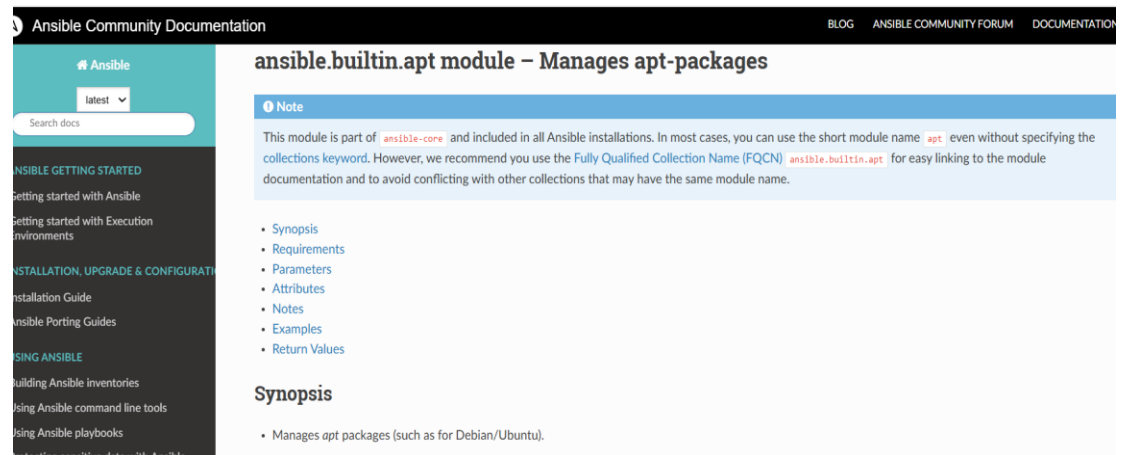


- Modules are well-structured, often under collections like `ansible.builtin`, `community.docker`, `amazon.aws`, etc.
- **Search Functionality:** Use the search bar to find specific modules (e.g., `yum`, `service`, `copy`).

- **Module Documentation Contents:**

- **Description:** Explains what the module does.
- **Parameters:** Lists all available arguments/options for the module, their types, and descriptions. This is crucial for understanding what inputs a module accepts.
- **Requirements:** Any prerequisites for the module on the target system (e.g., Python libraries).
- **Examples:** Practical playbook snippets demonstrating how to use the module with various arguments. Often the best place to start when learning a new module.

Lets look at an example of `apt` module under `ansible.builtin`



10. Projects

10.1 Project2 – Install Tomcat service in centos target machine and then copy customized index page from control node to target node

- **Tasks in Playbook**

Task1 – Install httpd using **yum module**

Task2 – Start httpd service using **service module**

Task3 - Copy index.html from the control node to /var/www/html/ on web servers using **copy module**

Copy Module: ansible.builtin.copy (found under "Files modules").

Key Parameters:

- src: Path to the source file on the **control node**.
- dest: Full path to the destination file on the **target node** (including filename). This is a **required** parameter.
- backup: yes (Optional): If the destination file already exists, it will be backed up with a timestamp before being overwritten. This is an example of a useful optional parameter found in documentation.

```

---
- name: Webserver setup
  hosts: webserver
  become: yes
  tasks:
    - name: Install httpd
      yum:
        name: httpd
        state: present

    - name: Start httpd service
      service:
        name: httpd
        state: started
        enabled: yes

    - name: Copy file with owner and permissions
      copy:
        src: files/index.html
        dest: /var/www/html/index.html
        backup: yes

```

- **Setup:** Create a files directory in the same location as your playbook, and place index.html inside it.

The project folder would contain these

```

vagrant@control:~/vprofile/ansible/ansible/exercise5$ ls
files  inventory  web.yml

```

Files folder would contain the customized index.html file

- **Run the pLaybook:**

```

vagrant@control:~/vprofile/ansible/ansible/exercise5$ ansible-playbook -i inventory web.yml
PLAY [webserver setup] *****
TASK [Gathering Facts] *****
ok: [web02]
ok: [web01]

TASK [Install httpd] *****
ok: [web01]
ok: [web02]

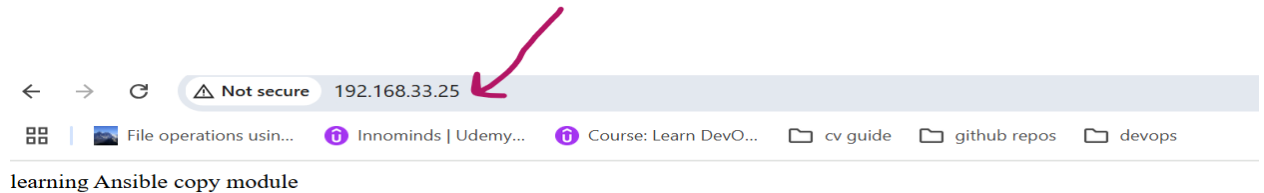
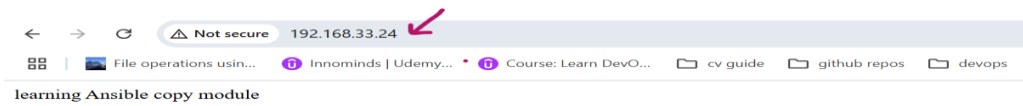
TASK [Start httpd service] *****
ok: [web02]
ok: [web01]

TASK [Copy file with owner and permissions] *****
ok: [web02]
ok: [web01]

PLAY RECAP *****
web01 : ok=4  changed=0  unreachable=0  failed=0  skipped=0  rescued=0  ignore
d=0
web02 : ok=4  changed=0  unreachable=0  failed=0  skipped=0  rescued=0  ignore

```

- **Verification:** After running the playbook, SSH into a target web server, navigate to /var/www/html/, and verify index.html and the timestamped backup file if backup: yes was used. Also we can check in the browser with corresponding server's ip address:80.



So, It can be seen for both the webserver's customized index.html replaced the tomcat home page

10.2 Project3 – This project installs mysql database in a target centos machine , starts the database service, creates a database and a user for the database

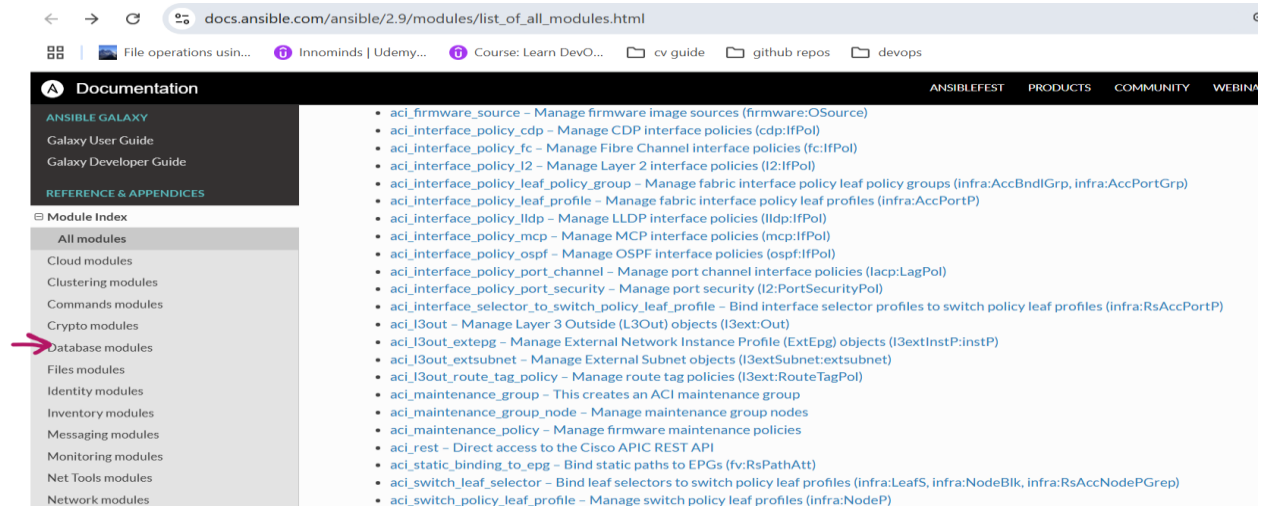
This session focuses on the practical workflow of a DevOps engineer: identifying the right Ansible module for a task, implementing it, and effectively troubleshooting issues that arise due to module dependencies or configuration.

- **Find the Module for the specific Task-** Search Ansible documentation for the appropriate module. Write the module in your playbook with the correct parameters.
- **Playbook & Tasks**
 - Task1** – Install MariaDB server using yum module
 - Task2** – Start the service using service module

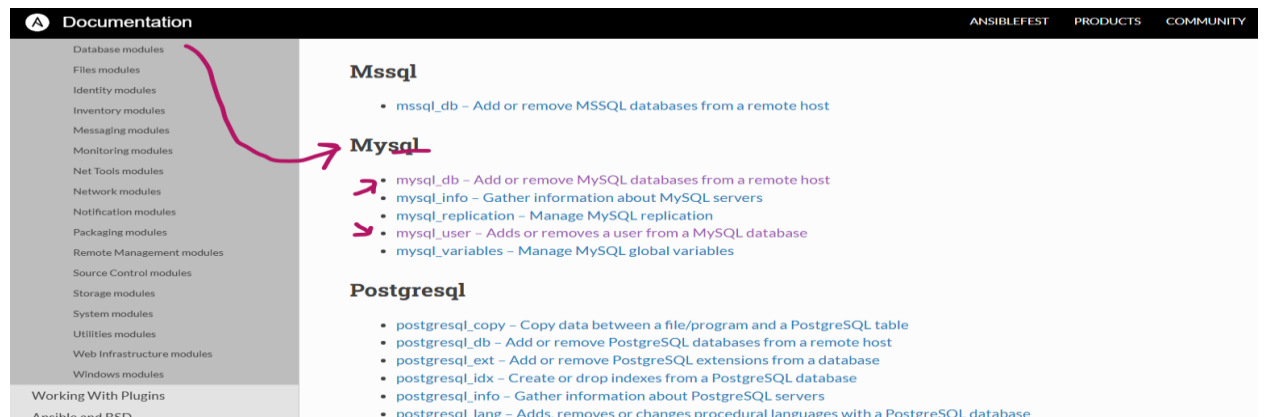
```
---
- name: DBserver setup
  hosts: dbservers
  become: yes
  tasks:
    - name: Install mariaDB-server
      yum:
        name: mariadb-server
        state: present

    - name: Start MariaDB service
      ansible.builtin.service:
        name: mariadb
        state: started
        enabled: yes
```


Task3 – Create a new Database using mysql_db module. For this task first we need to search for the module. How do we find the module? For that either go to this link https://docs.ansible.com/ansible/2.9/modules/list_of_all_modules.html => Database modules => mysql



Once we click on Database modules => we can see all the database modules including mysql , postgresql etc



Then in mysql we can see the exact modules for specific tasks , like for database management mysql_db, for user management mysql_user.

Remember mysql is under ansible community collection unlike other copy, apt are from ansible.builtin collection

Ansible Community Documentation

BLOG ANSIBLE COMMUNITY FORUM DOCUMENTATION

Ansible
latest

Search docs

ANSIBLE GETTING STARTED
Getting started with Ansible
Getting started with Execution environments

INSTALLATION, UPGRADE & CONFIGURATION
Installation Guide
Ansible Porting Guides

USING ANSIBLE
Building Ansible inventories
Using Ansible command line tools

community.mysql.mysql_db module – Add or remove MySQL or MariaDB databases from a remote host

Note

This module is part of the `community.mysql` collection (version 3.14.0).

You might already have this collection installed if you are using the `ansible` package. It is not included in `ansible-core`. To check whether it is installed, run `ansible-galaxy collection list`.

To install it, use: `ansible-galaxy collection install community.mysql`. You need further requirements to be able to use this module, see [Requirements](#) for details.

To use it in a playbook, specify: `community.mysql.mysql_db`.

- Synopsis
- Requirements
- Parameters
- Attributes

You can search for the module from this link also -

https://docs.ansible.com/ansible/latest/collections/index_module.html

docs.ansible.com/ansible/latest/collections/index_module.html

File operations using... Innominds | Udem... Course: Learn DevO... cv guide github repos devops

Ansible Community Documentation

BLOG ANSIBLE COMMUNITY FORUM

Unasked Questions

Reference: Module Utilities

Ansible Automation Platform

Information Hub

File output

Map

Roadmaps

- `community.mongodb.mongodb_shell` – Run commands via the MongoDB shell.
- `community.mongodb.mongodb_shutdown` – Cleans up all database resources and then terminates the mongod/mongos process.
- `community.mongodb.mongodb_status` – Validates the status of the replicaset.
- `community.mongodb.mongodb_stepdown` – Step down the MongoDB node from a PRIMARY state.
- `community.mongodb.mongodb_user` – Adds or removes a user from a MongoDB database

community.mysql

- `community.mysql.mysql_db` – Add or remove MySQL or MariaDB databases from a remote host
- `community.mysql.mysql_info` – Gather information about MySQL or MariaDB servers
- `community.mysql.mysql_query` – Run MySQL or MariaDB queries
- `community.mysql.mysql_replication` – Manage MySQL or MariaDB replication
- `community.mysql.mysql_role` – Adds, removes, or updates a MySQL or MariaDB role
- `community.mysql.mysql_user` – Adds or removes a user from a MySQL or MariaDB database
- `community.mysql.mysql_variables` – Manage MySQL or MariaDB global variables

community.network

Task4 – Create database user with username and password with database privileges

Remember to install the community.mysql collection on the control node first: check if the collection is present in your control node or not by running the command – **ansible-galaxy collection list**

```
community.aws 8.0.0
community.ciscosmb 1.0.9
community.crypto 2.22.3
community.digitalocean 1.27.0
community.dns 3.1.0
community.docker 3.13.3
community.general 9.5.2
community.grafana 1.9.1
community.hashi_vault 6.2.0
community.hrobot 2.0.2
community.library_inventory_filtering_v1 1.0.2
community.libvirt 1.3.0
community.mongodb 1.7.8
community.mysql 3.11.0
community.network 5.1.0
community.okd 3.0.1
community.postgresql 3.9.0
community.proxysql 1.6.0
community.rabbitmq 1.3.0
```

If not then run this command => **ansible-galaxy collection install community.mysql** to install

Finally the tasks look like

```
- name: Create a new database with name 'bobdata'
mysql_db:
  name: bobdata
  state: present
  login_unix_socket: /var/lib/mysql/mysql.sock
  become: yes

- name: Create database user with name 'bob' and password '12345' with all database privileges
mysql_user:
  name: bob
  password: 12345
  priv: '*,*:ALL'
  state: present
  login_unix_socket: /var/lib/mysql/mysql.sock
  become: yes
```

- **Run the playbook**
- **Troubleshooting Scenario:**

10.2.1 Error 1: Missing Python Dependencies:

- **Problem:** Running the mysql_db task initially fails with an error indicating missing Python dependencies (e.g., "Python MySQL libraries are required").

```
TASK [Create a new database with name 'bobdata']
*****
*****
fatal: [db01]: FAILED! => {"changed": false, "msg":
"unable to find /root/.my.cnf. Exception message: (1698,
\'Access denied for user \'root\'@\'localhost\'\)\'"}
^
```

- **Reason:** The module's Requirements section (often overlooked initially) clearly states that a Python MySQL library (like PyMySQL or mysql-python) is needed on the **target host**.

mysql_db – Add or remove MySQL databases from a remote host

- Synopsis
- Requirements
- Parameters
- Notes
- Examples
- Return Values
- Status

Synopsis

- Add or remove MySQL databases from a remote host.

Requirements

The below requirements are needed on the host that executes this module.

- MySQLdb (Python 2.x)
- PyMySQL (Python 2.7 and Python 3.X), or
- mysql (command line binary)
- mysqldump (command line binary)

o Solution:

- SSH into a target DB machine (db01).
- Use `yum search python | grep -i mysql` (for CentOS) to find available Python MySQL packages.
- Identify the correct package (e.g., `python3-PyMySQL`).

```
[vagrant@db01 ~]$ yum search python | grep -i mysql
Last metadata expiration check: 1 day, 19:38:36 ago on Fri 11 Jul 2025 10:27:46 AM UTC.
python3-PyMySQL.noarch : Pure-Python MySQL client library
python3.11-PyMySQL.noarch : Pure-Python MySQL client library
python3.11-PyMySQL+rsa.noarch : Metapackage for python3.11-PyMySQL: rsa extras
python3.12-PyMySQL.noarch : Pure-Python MySQL client library
python3.12-PyMySQL+rsa.noarch : Metapackage for python3.12-PyMySQL: rsa extras
```

- Add a task to the **playbook** to install this dependency using `ansible.builtin.yum` (or `ansible.builtin.pip` if using `pip` for Python packages) *before* the `mysql_db` task.

```
- name: Install PyMySQL dependency
  ansible.builtin.yum:
    name: python3-PyMySQL # Or other specific package name
    state: present

- name: Create a new database with name 'bobdata'
  mysql_db:
    name: bobdata
    state: present
    login_unix_socket: /var/lib/mysql/mysql.sock
    become: yes
```

10.2.2 Error 2: Socket File Location / Access Denied:

- **Problem:** After installing dependencies, the mysql_db task might still fail with "Access denied" or "unable to find socket file" errors.
- **Reason:** The MySQL/MariaDB server and the Python client might not be correctly configured to connect via the default Unix socket. The error message often points to an incorrect socket path or that it's missing.
- **Solution:**
 - **Consult Google/Stack Overflow:** Searching the specific error message (e.g., "Ansible mysql_db access denied socket file") is a common troubleshooting step.
 - **Identify Correct Socket Path:** For CentOS, the common MariaDB socket path is often /var/lib/mysql/mysql.sock.
 - **Use login_unix_socket Parameter:** The mysql_db and mysql_user modules accept a login_unix_socket parameter to explicitly specify the socket file path.

```
- name: Create a new database with name 'bobdata'
  mysql_db:
    name: bobdata
    state: present
    login_unix_socket: /var/lib/mysql/mysql.sock
    become: yes
```

- Verification – Once the playbook runs successfully check in DB01 whether the database service installed or not

```
[vagrant@db01 ~]$ systemctl status mariadb
● mariadb.service - MariaDB 10.5 database server
   Loaded: loaded (/usr/lib/systemd/system/mariadb.service; enabled; preset: disabled)
   Active: active (running) since Sun 2025-07-13 03:48:17 UTC; 2h 41min ago
     Docs: man:mariadb(8)
           https://mariadb.com/kb/en/library/systemd/
   Process: 555 ExecStartPre=/usr/libexec/mariadb-check-socket (code=exited, status=0/SUCCESS)
   Process: 611 ExecStartPre=/usr/libexec/mariadb-prepare-db-dir mariadb.service (code=exited, status=0/SUCCESS)
   Process: 749 ExecStartPost=/usr/libexec/mariadb-check-upgrade (code=exited, status=0/SUCCESS)
  Main PID: 676 (mariabdd)
    Status: "Taking your SQL requests now..."
     Tasks: 8 (limit: 2681)
    Memory: 76.3M
       CPU: 1.570s
    CGroup: /system.slice/mariadb.service
            └─676 /usr/libexec/mariabdd --basedir=/usr
```

Then check database has been created or not – use mysql client to login into mariadb

sudo mysql -u bob -p

it would prompt for password => give the password (12345 in this case)

If run successfully we are inside mariadb DBMS

```
[vagrant@db01 ~]$ sudo mysql -u bob -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 6
Server version: 10.5.27-MariaDB MariaDB Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement

MariaDB [(none)]> |
```

Common Commands once inside the MariaDB shell:

- SHOW DATABASES; - List all databases.
- USE database_name; - Select a database to work with.
- SHOW TABLES; - List tables in the currently selected database.
- DESCRIBE table_name; - Show the structure of a table.
- SELECT * FROM table_name; - Query data from a table.
- EXIT; or QUIT; - Exit the MariaDB shell.

```
MariaDB [(none)]> show databases;
+-----+
| Database |
+-----+
| bobdata  |
| information_schema |
| mysql    |
| performance_schema |
+-----+
4 rows in set (0.004 sec)
```

So, it can be seen that our database bobdata has been successfully created in MariaDB with user Bob

11. Ansible configuration

In this section we explore Ansible's configuration settings, detailing how to override default behaviors and manage configurations across different levels of precedence.

11.1 What is ansible configuration?

Till now we are running ansible playbooks successfully, means ansible using default configuration. Like ansible control machine logs into the target machine at port 22.

11.2 Why Configure Ansible?

- **Changing Defaults:** Ansible operates with sensible defaults (e.g., SSH on port 22, no logging). Now suppose security purpose ssh to target machine changed to 2022 instead of 22. so these default configurations don't meet the needs. At that time we need to change Ansible's configuration.

11.3 Ansible Configuration File Precedence (Order of Priority)

Ansible checks for configuration settings in a specific order, with settings found earlier in the list overriding those found later:

1. **ANSIBLE_CONFIG environment variable:**

- **Highest Priority.**
- If this environment variable is set and points to a file, Ansible will use that file as its primary configuration.
- Example: `export ANSIBLE_CONFIG=/path/to/my_ansible.cfg`

2. **ansible.cfg in the current working directory:**

- **Recommended for Project-Specific Configuration.**
- If an `ansible.cfg` file exists in the directory where you run your `ansible` or `ansible-playbook` command, it will be used.
- This is ideal for repository-specific settings, ensuring consistent behavior for all team members working on that project.

3. **~/ansible.cfg (in your home directory):**

- **User-Specific Configuration.**
- Create A hidden file in your user's home directory. Useful for personal defaults that apply to all your Ansible projects unless overridden by a project-specific `ansible.cfg`.

4. **/etc/ansible/ansible.cfg:**

- **Lowest Priority (Global System-Wide Configuration).**
- The default global configuration file, typically installed with Ansible. This affects all Ansible operations on that system unless overridden by higher-priority files or variables.

ANSIBLE_CONFIG environment variable(system level) > ansible.cfg(Project level) > ~/ansible.cfg(system level) > /etc/ansible/ansible.cfg(system level)

Order of Ansible Config

1. **ANSIBLE_CONFIG** (environment variable if set)
2. **ansible.cfg** (in the current directory)
3. **~/.ansible.cfg** (in the home directory)
4. **/etc/ansible/ansible.cfg**

11.4 Exploring the Global ansible.cfg (/etc/ansible/ansible.cfg)

- We would explore the global configuration file to understand different configuration properties of an ansible configuration file.
- **Generate the ansible.cfg global file** - Initially it remains empty, just contains the instruction to generate the config file.

- ✓ in the home directory only we would switch to root user.
- ✓ Then go inside /etc/ansible
- ✓ Check the content of default ansible.cfg file

```
# Since Ansible 2.12 (core):
# To generate an example config file (a "disabled" one with all default settings, commented out):
# $ ansible-config init --disabled > ansible.cfg
#
# Also you can now have a more complete file by including existing plugins:
# ansible-config init --disabled -t all > ansible.cfg
#
# For previous versions of Ansible you can check for examples in the 'stable' branches of each version
# Note that this file was always incomplete and lagging changes to configuration settings
# for example, for 2.9: https://github.com/ansible/ansible/blob/stable-2.9/examples/ansible.cfg
```

- ✓ To generate the configuration rename the existing ansible.cfg as ansible.cfg_copy
- ✓ Then run the command mentioned the file –
ansible-config init --disabled > ansible.cfg

- ✓ Ansible.cfg is generated

```
vagrant@control:~/vprofile/ansible/ansible$ ls /etc/ansible
ansible.cfg  ansible.cfg_copy  hosts  roles
```

- ✓ Vim ansible.cfg => check the content of the file

- This is a large file (often over 500 lines) with various sections.
- **[defaults] section:** Contains the most common and frequently used settings.
- **Other Sections:** [privilege_escalation], [ssh_connection], [inventory], [vault], etc.

- **Comments:** Lines starting with # or ; are comments.
- **Key Settings Examples:**
 - **host_key_checking = False:** (Previously set) Disables strict SSH host key checking (useful in dynamic Vagrant environments).
 - **ask_pass = False:** By default, Ansible won't ask for SSH passwords (it relies on keys). Set to True if you want it to prompt.
 - **forks = 5:** Controls the number of parallel SSH connections Ansible makes to target hosts at a time. Adjust based on your control machine's resources (CPU, RAM, network bandwidth) and target server capacity.
 - **inventory = /path/to/inventory_file:** Specifies the default inventory file path, so you don't need to use -i every time.
 - **log_path = /path/to/ansible.log:** Enables logging of all Ansible executions to the specified file. By default, no logs are kept.

```
# (boolean) This controls whether an Ansible playbook should prompt for a login password. If u
authentication, you probably do not need to change this setting.
;ask_pass=False

# (boolean) This controls whether an Ansible playbook should prompt for a vault password.
;ask_vault_pass=False

# (paths spec) Colon-separated paths in which Ansible will search for Cache Plugins.
;cache_plugins=/root/.ansible/plugins/cache:/usr/share/ansible/plugins/cache

# (paths spec) Colon-separated paths in which Ansible will search for Callback Plugins.
```

11.5 Creating a Project-Specific ansible.cfg (Recommended Practice)

- **Location:** Create an ansible.cfg file directly in your project directory (where your playbooks and inventory are).

```
vagrant@control:~/vprofile/ansible/ansible/exercise7$ ls
ansible.cfg db.yml inventory
```

- **Contents (Example):**

```
vagrant@control:~/vprofile/ansible/ansible/exercise7$ cat ansible.cfg
[defaults]
host_key_checking=false
inventory=./inventory
forks = 5
log_path = /var/log/ansible.log

[privilege escalation]
become=True
become_method=sudo
become_ask_pass=False
```

- **Run the Playbook** – ansible-playbook db.yml. No need to mention the inventory file path, since it is already mentioned in the configuration file
- **Debugging- Managing Log File Permissions** - If you set log_path to a system-wide location like /var/log/ansible.log, the vagrant user (or whichever user is running Ansible) might not have write permissions.

Solution:

```
sudo touch /var/log/ansible.log
```

```
sudo chown vagrant:vagrant /var/log/ansible.log
```

```
# Change ownership to the user running Ansible
```

Then we execute the playbook in verbose mode => ansible-playbook db.yml -vv; we would be able to see the detailed logs stored in /var/log/ansible.log file

```
vagrant@control:~/vprofile/ansible/ansible/exercise7$ cat /var/log/ansible.log
2025-07-14 04:15:34,843 p=3782 u=vagrant n=ansible | PLAY [DBserver setup]
2025-07-14 04:15:34,853 p=3782 u=vagrant n=ansible | TASK [Gathering Facts]
2025-07-14 04:15:37,035 p=3782 u=vagrant n=ansible | ok: [db01]
2025-07-14 04:15:37,044 p=3782 u=vagrant n=ansible | TASK [Install mariaDB-server]
2025-07-14 04:15:39,967 p=3782 u=vagrant n=ansible | ok: [db01]
2025-07-14 04:15:39,975 p=3782 u=vagrant n=ansible | TASK [Start MariaDB service]
2025-07-14 04:15:41,388 p=3782 u=vagrant n=ansible | ok: [db01]
2025-07-14 04:15:41,395 p=3782 u=vagrant n=ansible | TASK [Create a new database with name 'bobdata']
2025-07-14 04:15:42,334 p=3782 u=vagrant n=ansible | ok: [db01]
2025-07-14 04:15:42,341 p=3782 u=vagrant n=ansible | TASK [Create database user with name 'bob' and password
2345' with all database privileges]
2025-07-14 04:15:43,307 p=3782 u=vagrant n=ansible | [WARNING]: Option column_case_sensitive is not provided.
he default is now false, so the column's name will
be uppercased. The default will be changed to true in community.mysql 4.0.0.
2025-07-14 04:15:43,308 p=3782 u=vagrant n=ansible | ok: [db01]
2025-07-14 04:15:43,311 p=3782 u=vagrant n=ansible | PLAY RECAP
```

11.6 Conclusion

- **Always use a project-specific ansible.cfg in your repository.**

- Don't try to memorize all settings; instead, understand the **concept of configuration** and how to use the **documentation** to find specific settings when needed. Link below for documentation-

https://docs.ansible.com/archive/ansible/2.4/intro_configuration.html

- Configuration changes allow you to adapt Ansible's powerful automation capabilities to the unique requirements of your infrastructure.

12. Ansible Variables

- **Purpose:** Variables in Ansible, like in other programming languages (Bash, Python), allow you to store and reuse values. This makes playbooks more flexible, reusable, and easier to maintain by separating data from logic.

- **Types:**

Custom Variables (User-Defined): Variables that you create yourself.

In-Playbook: Defined using the vars keyword within a playbook.

External Files: Stored in dedicated files for better organization and reusability.

Built-in Variables (Ansible-Generated): Variables that Ansible creates automatically.

Fact Variables: Information collected about the target host.

Registered Variables: The output of a task that is stored for later use.



12.1 Playbook Custom Variables :- Defining variables directly in the playbook

- **Define the variable** – In the playbook variables are declared under the vars: keyword at the same columns as hosts and tasks

Syntax:-

```
---
- name: DBserver setup
  hosts: dbservers
  become: yes
  vars:
    db_name: students
    db_user: sia
    db_pass: 12345

  tasks:
```

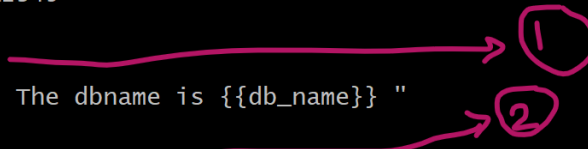
- **Using Variables in Playbooks** - To use a variable within your playbook, you must enclose its name in double curly braces "{{ }}" . When the variable is used as a value in a key-value pair, it should also be enclosed in double quotes. Syntax - "{{ variable_name }}"

```
- name: Create a new database
  mysql_db:
    name: "{{ db_name }}"
    state: present
    login_unix_socket: /var/lib/mysql/mysql.sock
    become: yes
```

- **Print the Variable using debug module** - The debug module is a vital tool for troubleshooting and understanding playbook execution by printing information to the console. It's analogous to echo in Bash or print in Python. There are two ways we can print the variable. One is how we use the variable inside a message and second is direct

```
become: yes
vars:
  db_name: students
  db_user: sia
  db_pass: 12345

tasks:
  - debug:
      msg: " The dbname is {{db_name}} "
  - debug:
      var: db_user
```



12.2 Registered variable

- The **register** keyword is used to capture the JSON output of a task and store it into a variable
- It is being written in the same column as module
- Add register: <variable_name> to any task
- Example

```
- name: Create database user with all database privileges
mysql_user:
  name: "{{ db_user }}"
  password: "{{ db_pass }}"
  priv: ' *.*:ALL'
  state: present
  login_unix_socket: /var/lib/mysql/mysql.sock
  become: yes
  register: db_user_output
- name: print db_user_output variable
  debug:
    var: db_user_output
```

- Here we registering output of the task in a variable named db_user_output. Then we are printing the output using debug module.

12.3 variables defined outside the playbook - For better reusability and organization, Ansible variables should primarily be defined outside the playbook in dedicated files

- **group_vars/all** - Variables defined in this file apply to **all hosts** listed in the Ansible inventory. Here group_vars is the folder and all is a file created under group_vars
- **group_vars/<group_name>** (e.g., group_vars/webserver) - Variables defined here apply **only to hosts belonging to the specified group**
- **host_vars/<host_name>** (e.g., host_vars/web02) - Variables defined here apply **only to the specific host** matching the filename
- Example structure

```
vagrant@control:~/vprofile/ansible/ansible/exercise9$ tree
.
├── ansible.cfg
├── group_vars
│   ├── all
│   └── webserver
├── host_vars
│   ├── db01
│   └── web02
├── inventory
├── print_facts.yml
├── var_precedence.yml
└── variables_list_dic.yml
```

12.4 Command line variable – while running the playbook we pass the variable and its value. But the variable must be defined somewhere either in the playbook or any variables file

```
ansible-playbook -e "my_var=value"
```

12.5 Variable Precedence

Command line(-e) >> Playbook_variable(vars) >> host_vars/<host_name> >> group_vars/<group_name> >> group_vars/all

12.6 Ansible Fact Variables

- **Definition:** Fact variables are dynamic variables that Ansible automatically gathers about your target hosts. They provide comprehensive information about the host's operating system, hardware, network configuration, etc.

Fact Variables: setup module		
ansible_os_family OS name like RedHat, Debian etc	ansible_processor_cores Number of CPU cores	ansible_kernel Kernel Version.
ansible_devices Connected Device Information	ansible_default_ipv4 IP, MAC address, gateway etc.	ansible_architecture 64 bit or 32 bit

- **How they are generated:**
 - Fact variables are generated by the **ansible.builtin.setup** module
 - By default, every Ansible playbook implicitly runs a "**Gathering Facts**" task at the beginning of each play, which executes the setup module

```
PLAY [print facts] *****
TASK [Gathering Facts] *****
ok: [web01]
ok: [web02]
ok: [db01]
```

- These facts are available only at runtime and are not permanently stored
- To see the fact variables of a server, we can manually run ad-hoc command with setup module => **ansible -m setup web01**

```
vagrant@control:~/vprofile/ansible/ansible/exercise9$ ansible -m setup web01
web01 | SUCCESS => {
  "ansible_facts": {
    "ansible_all_ipv4_addresses": [
      "10.0.2.15",
      "192.168.33.24"
    ],
    "ansible_all_ipv6_addresses": [
      "fd00::a00:27ff:fee4:ec85",
      "fe80::a00:27ff:fee4:ec85",
      "fe80::a00:27ff:fe64:89e7"
    ],
    "ansible_apparmor": {
      "status": "disabled"
    },
    "ansible_architecture": "x86_64",
    "ansible_bios_date": "12/01/2006",
    "ansible_bios_vendor": "innotek GmbH",
    "ansible_bios_version": "VirtualBox",
    "ansible_board_asset_tag": "NA",
    "ansible_board_name": "VirtualBox",
    "ansible_board_serial": "NA",
    "ansible_board_vendor": "Oracle Corporation",
    "ansible_board_version": "1.2",
    "ansible_chassis_asset_tag": "NA",
    "ansible_chassis_serial": "NA",
```

- **Accessing Fact Variables in playbook**

- In Playbook Fact variables are accessed using the same Jinja2 syntax as other variables: `{{ variable_name }}`. they can be printed just using `var` also.
- **Example1: Simple String like `ansible_distribution`**

```
{
  "ansible_distribution": "CentOS",
  "ansible_distribution_file_parsed": true,
  "ansible_distribution_file_path": "/etc/centos-release",
  "ansible_distribution_file_variety": "CentOS",
```

```
---
- name: print facts
  hosts: all
  tasks:
    - name: Print os name
      debug:
        var: ansible_distribution
```

- **Example2: Nested Dictionary (Memory Information)**

Json config
<pre> "ansible_memory_mb": { "nocache": { "free": 257, "used": 199 }, "real": { "free": 51, "total": 456, "used": 405 }, "swap": { "cached": 0, "free": 1023, "total": 1023, "used": 0 } } </pre>
In Playbook
<pre> - name: Print Ram Memory debug: var: ansible_memory_mb.real.free </pre>

- Example3: List (Processor Information)

Json Config
<pre> "ansible_processor": ["0", "AuthenticAMD", "AMD Ryzen 7 7730U with Radeon Graphics"], </pre>
Playbook
<pre> - name: Print Processor name debug: var: ansible_processor[2] </pre>

13. Conditions & loops

13.1 Condition

- Execute tasks only when specific criteria are met
- Syntax: **when:** <condition> add in the task
- When keyword is added at the same indentation level as the module name within a task
- In the condition we can add fact variables checking condition. Operators use == for equality, != for inequality, >/</>= /<= for comparisons, and and/or for combining conditions.
- Example


```

- name: Start chronyd service on CentOS
  ansible.builtin.service:
    name: chronyd
    state: started
    enabled: yes
  when: ansible_distribution == "CentOS"

- name: Start ntp service on Ubuntu
  ansible.builtin.service:
    name: ntp
    state: started
    enabled: yes
  when: ansible_distribution == "Ubuntu"

```

13.2 loops

- Example – Install multiple packages using loop

```

- name: Install NTP agent on CentOS
  ansible.builtin.yum:
    name: "{{ item }}" # 'item' holds the current package name in each iteration
    state: present
  when: ansible_distribution == "CentOS"
  loop:
    - chrony
    - wget
    - git
    - zip
    - unzip

```

- Loop keyword is added to the task
- It is added at the same indentation level as the module name
- **Loop with list** - If The value of loop: is a list of items to iterate over. Each item in the list is accessed via the special variable `{{ item }}`
- **Loop with dictionary** - If the value of loop: is dictionary, then each item in the dic is accessed via `{{ item.key_name }}`.

```

- name: Add multiple users
  ansible.builtin.user:
    name: "{{ item.name }}"
    groups: "{{ item.groups }}" # Accessing dictionary keys using dot notation
    state: present
  loop:
    - { name: 'testuser1', groups: 'wheel' }
    - { name: 'testuser2', groups: 'devs' }
    - { name: 'testuser3', groups: 'ops' }

```

14. Copy, file module & template module

14.1 Copy module

- Full name - **ansible.builtin.copy**. This module is part of ansible-core and included in all Ansible installations
- The ansible.builtin.copy module copies a file or a directory structure from the local or remote machine to a location on the remote machine. File system meta-information (permissions, ownership, etc.) may be set, even when the file or directory already exists on the target system. Some meta-information may be copied on request.
- Example :

- **name:** Copy file with owner and permissions

ansible.builtin.copy:

src: /srv/myfiles/foo.conf

dest: /etc/foo.conf

owner: foo

group: foo

mode: '0644'

- **name:** Copy using inline content

ansible.builtin.copy:

content: '# This file was moved to /etc/other.conf'

dest: /etc/mine.conf

- Practical usage:-
If we want to update the default apache web page to a customized html page.

- **name:** Copy file with owner and permissions

copy:

src: files/index.html

dest: /var/www/html/index.html

backup: yes

In this task, control machine would replace the default index.html of remote server to the customized html page

14.2 File module

- [Fully Qualified Collection Name \(FQCN\)](#) - **ansible.builtin.file**. This module is part of ansible-core and included in all Ansible installations.

- Set attributes of files, directories, or symlinks and their targets. Alternatively, remove files, symlinks or directories.
- Many other modules support the same options as the [ansible.builtin.file](#) module - including [ansible.builtin.copy](#), [ansible.builtin.template](#), and [ansible.builtin.assemble](#)
- Example:

```
- name: Create a directory if it does not exist
  ansible.builtin.file:
    path: /etc/some_directory
    state: directory
    mode: '0755'
```

```
- name: Remove file (delete file)
  ansible.builtin.file:
    path: /etc/foo.txt
    state: absent
```

14.3 Template Module

- **Fully Qualified Collection Name (FQCN)** - **ansible.builtin.template** . This module is part of ansible-core and included in all Ansible installations.
- Templates are processed by the [Jinja2 templating language](#).
- Example:

```
- name: Template a file to /etc/file.conf
  ansible.builtin.template:
    src: /mytemplates/foo.j2
    dest: /etc/file.conf
    owner: bin
    group: wheel
```

- Practical Example:- update the configuration of apache2 using a jinja2 template in control machine

Step1 – write the task in playbook using template module

```
- name: Write the apache config file
  ansible.builtin.template:
    src: templates/httpd.j2
    dest: /etc/httpd.conf
  notify:
    - Restart apache
```

Step2 – create the project structure for the template file

For this code there should be a folder named templates and a template named httpd.j2 is there

Sample project structure would be:-

```
.
├── ansible.cfg
├── group_vars
│   ├── all
│   └── webservers
├── handler.yml
├── host_vars
│   └── web03
├── inventory
├── templates
│   ├── apache.conf.j2
│   └── httpd.j2
└── verify-apache.yml
```

Step 3 – Write the jinja2 template file.

Now how to write this jinja2 templates mentioned In link before. Better take an existing configuration file and make changes in it.

Sample J2 file

```
# Apache Configuration Template
Listen {{ http_port }}
#ServerAdmin webmaster@localhost
MaxClients {{ max_clients }}
```

Step 4 – Declare the values of the variables mentioned in jinja2 template.

Now the variables mentioned in the template, can have their values group_vars file/host_vars/ in playbook

15. Handlers

15.1 Definition - Ansible **handlers**, a special type of task that runs only when explicitly "notified" by another task, typically after a change has occurred.

15.2 Problem solved by handler - Unnecessary Service Restarts: Without handlers, if you have a task to deploy a configuration file and a separate task to restart a service, the service will restart every time the playbook runs, even if the configuration file hasn't changed. This is inefficient and can cause unnecessary downtime.

15.3 Example – Lets understand it with an example

```
- name: Install nginx on web servers
  hosts: web04
  become: true
  vars:
    nginx_worker_processes: 2
    nginx_listen_port: 5001

  tasks:
    - name: Install nginx in ubuntu machines
      apt:
        name: nginx
        state: present
        update_cache: yes
        when: ansible_os_family == 'Debian'

    - name: Install nginx in centos machines
      yum:
        name: nginx
        state: present
        update_cache: yes
        when: ansible_os_family == 'RedHat'

    - name: update the configuration of nginx server
      template:
        src: ./templates/nginx.conf.j2
        dest: /etc/nginx/sites-available/default
      notify:
        - restart nginx

    - name: start and enable nginx
      service:
        name: nginx
        state: started
        enabled: yes

  handlers:
    - name: restart nginx
      service:
        name: nginx
        state: restarted
```

So, in this playbook – we are installing the nginx server on web04 , changing the configuration of nginx and restart the nginx server only when the configuration is changed. **We are trying to change the configuration of nginx server from port 80 to 5000.**

Folder structure

```
vagrant@control:~/vprofile/ansible/ansible/exercise15$ tree
.
├── ansible.cfg
├── inventory
├── nginx_setup.yml
├── templates
│   └── nginx.conf.j2
└── 1 directory, 4 files
```

Jinja template

```
vagrant@control:~/vprofile/ansible/ansible/exercise15$ cat templates/nginx.conf.j2
# Managed by Ansible - DO NOT EDIT MANUALLY
user www-data;
worker_processes {{ nginx_worker_processes }};

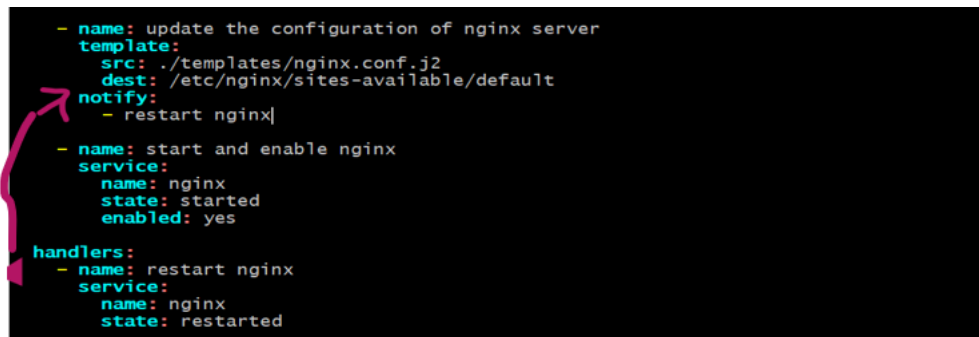
events {
    worker_connections 1024;
}

http {
    server {
        listen {{ nginx_listen_port }};
        server_name localhost;

        location / {
            root /var/www/html;
            index index.html;
        }
    }
}
```

15.3.1 How Handlers work

- Define the handler - Handlers are defined in a separate section of your playbook, typically at the same level as tasks, using the handlers: keyword.
- Notification - A regular task "notifies" a handler by using the notify: keyword. The value of notify: is the exact name of the handler task. As we are seeing in this code. The handler is restart nginx which is being notified when the configuration change task is true



```
- name: update the configuration of nginx server
  template:
    src: ./templates/nginx.conf.j2
    dest: /etc/nginx/sites-available/default
  notify:
    - restart nginx

- name: start and enable nginx
  service:
    name: nginx
    state: started
    enabled: yes

handlers:
  - name: restart nginx
    service:
      name: nginx
      state: restarted
```

- When a task with a notify statement runs, Ansible checks if that task reported a "changed" status (i.e., changed: true in its output). If the task *did* change something, Handler would be executed. In our case we are trying to update the default port to 5000 from port 80. Handlers are executed **only once** at the end of a play, after all regular tasks have completed. If the notifying task did *not* change anything (changed: false), the handler is not queued and will not execute

Handler got executed bcz the configuration task changed and got executed at last

```
vagrant@control:~/vprofile/ansible/ansible/exercise15$ ansible-playbook nginx_setup.yml

PLAY [Install nginx on web servers] *****
TASK [Gathering Facts] *****
ok: [web04]

TASK [Install nginx in ubuntu machines] *****
ok: [web04]

TASK [Install nginx in centos machines] *****
skipping: [web04]

TASK [update the configuration of nginx server] *****
changed: [web04]

TASK [start and enable nginx] *****
ok: [web04]

RUNNING HANDLER [restart nginx] *****
changed: [web04]

PLAY RECAP *****
web04 : ok=5 changed=2 unreachable=0 failed=0 skipped=1 rescued=0 ignored=0
```

Handler did not get executed bcz the configuration task has no change

```
vagrant@control:~/vprofile/ansible/ansible/exercise15$ ansible-playbook nginx_setup.yml

PLAY [Install nginx on web servers] *****
TASK [Gathering Facts] *****
ok: [web04]

TASK [Install nginx in ubuntu machines] *****
ok: [web04]

TASK [Install nginx in centos machines] *****
skipping: [web04]

TASK [update the configuration of nginx server] *****
ok: [web04]

TASK [start and enable nginx] *****
ok: [web04]

PLAY RECAP *****
web04 : ok=4 changed=0 unreachable=0 failed=0 skipped=1 rescued=0 ignored=0
```

15.3.2 Common Use Case: Restarting services after configuration file changes (often combined with `ansible.builtin.template` or `ansible.builtin.copy` modules)

Beyond Service Restarts: Handlers can be any task. Examples:

- Reloading a firewall after rules change.
- Triggering a database migration after schema updates.
- Copying a file after a user is created.
- Running a post-deployment script

16. Role

16.1 Why Use Ansible Roles?

- **Simplifies Playbook Management:** As playbooks grow (with variables, tasks, templates, files, handlers), managing everything in a single place becomes hard.

- **Encourages Reusability & Modularity:** Roles package configuration and logic in a reusable format, so you can share and reuse across projects/environments.

16.2 What Is an Ansible Role?

- Roles let you automatically load related vars, files, tasks, handlers, and other Ansible artifacts based on a known file structure. After you group your content into roles, you can easily reuse them and share them with other users.
- Typical use cases:
 - Specific roles: MySQL, Tomcat, Build Server – applied only to some machines.

16.3 Role Directory Structure

An Ansible role has a defined directory structure with seven main standard directories. You must include at least one of these directories in each role. You can omit any directories the role does not use. For example:

```
roles/
  common/          # this hierarchy represents a "role"
    tasks/         #
      main.yml      # <-- tasks file can include smaller files if warranted
    handlers/      #
      main.yml      # <-- handlers file
    templates/     # <-- files for use with the template resource
      ntp.conf.j2   # <----- templates end in .j2
    files/         #
      bar.txt       # <-- files for use with the copy resource
      foo.sh        # <-- script files for use with the script resource
    vars/          #
      main.yml      # <-- variables associated with this role
    defaults/      #
      main.yml      # <-- default lower priority variables for this role
    meta/          #
      main.yml      # <-- role dependencies
    library/       # roles can also include custom modules
    module_utils/  # roles can also include custom module_utils
    lookup_plugins/ # or other types of plugins, like lookup in this case

  webtier/         # same kind of structure as "common" was above, done for the webtier role
  monitoring/      # ""
  fooapp/          # ""
```

- By default, Ansible will look in most role directories for a main.yml file for relevant content
- Only put relevant content in each folder: Templates go in templates/, handlers in handlers/, etc.
- Defaults vs Vars: Variables in defaults/main.yml have lower priority than those in vars/main.yml. Usually, organization-wide default values go in defaults/.

16.4 How to Convert a Playbook into a Role

1. **Create the roles directory:** - `mkdir roles`
initiate roles directory structure - `ansible-galaxy init <role_name>`
2. Start with a working playbook. **Move playbook components:**
 - Tasks → roles/<role_name>/tasks/main.yml
 - Handlers → roles/<role_name>/handlers/main.yml

- Files/templates → respective folders
- Variables → usually defaults/main.yml

```
vagrant@control1:~/vprofile/ansible/ansible/exercise13$ tree
.
├── ansible.cfg
├── inventory
├── roles
│   └── apache_install
│       ├── README.md
│       ├── defaults
│       │   └── main.yml
│       ├── files
│       │   └── myfile.txt
│       ├── handlers
│       │   └── main.yml
│       ├── meta
│       │   └── main.yml
│       ├── tasks
│       │   └── main.yml
│       ├── templates
│       │   └── httpd.j2
│       ├── tests
│       │   ├── inventory
│       │   └── test.yml
│       └── vars
│           └── main.yml
└── verify-apache.yml
```

So, here we have created a folder named roles, under that created a role named apache_install using ansible-galaxy command.

3. In your main playbook:- in this case verify-apache.yml

Replace explicit lists of tasks,handlers,vars with a simple roles block:

Main playbook after applying role

```
vagrant@control1:~/vprofile/ansible/ansible/exercise13$ cat verify-apache.yml
---
- name: Verify apache installation
  hosts: web01
  become: yes
  roles:
    - apache_install
```

Main playbook before applying role. There was vars, tasks, handlers, templates all in one playbook

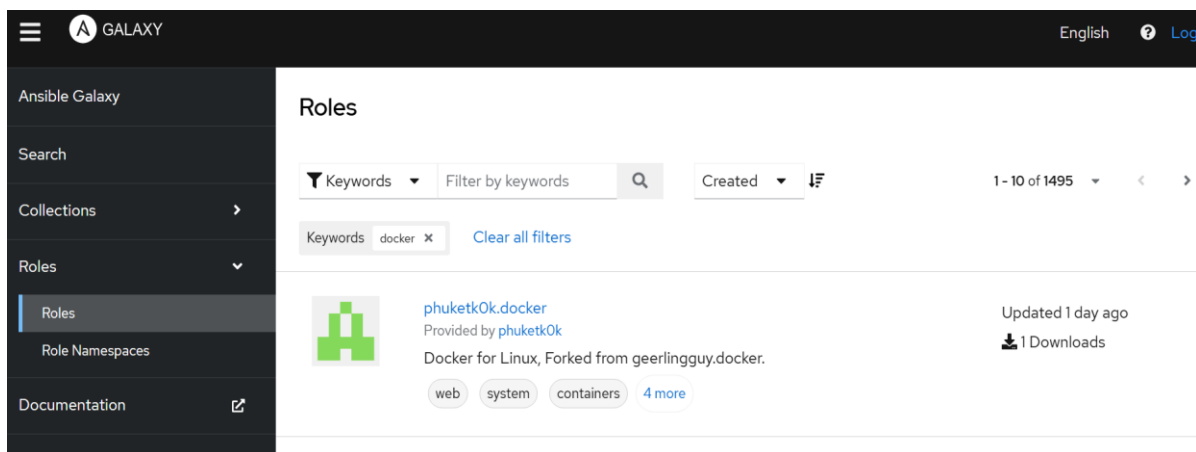
```
vagrant@control1:~/vprofile/ansible/ansible/exercise12$ cat verify-apache.yml
---
- name: Verify apache installation
  hosts: web01
  become: yes
  vars:
    http_port: 80
    max_clients: 200
  tasks:
    - name: Ensure apache is at the latest version
      ansible.builtin.yum:
        name: httpd
        state: latest
    - name: Write the apache config file
      ansible.builtin.template:
        src: templates/httpd.j2
        dest: /etc/httpd.conf
      notify:
        - Restart apache
    - name: Ensure apache is running
      ansible.builtin.service:
        name: httpd
        state: started
  handlers:
    - name: Restart apache
      ansible.builtin.service:
        name: httpd
        state: restarted
```

16.5 Benefits of Roles

- Cleaner Directory Structure
- Easy to Change/Access
- Highly Modular
- Reusable Across Projects/Environments
- Variables can be overridden per use-case
- Ansible community documentation for roles - https://docs.ansible.com/ansible/latest/playbook_guide/playbooks_reuse_roles.html

16.6 Ansible Galaxy Roles

- Ready-made Roles: Download with `ansible-galaxy install <role_name>`.
E.g., `geerlingguy.java`
- Customize by Overriding Variables:
Use the `vars` block in your playbook when invoking the role.
- Good Example Structure:
Study community roles to learn effective structuring and OS-specific task separation (include_tasks with conditionals).



Final Advice

- Write playbooks first—convert working solutions to roles for maintainability and reuse.
- Explore and study Ansible Galaxy roles but build your own if you need control or heavy customization.

17. Ansible for AWS Cloud Automation

This note summarizes how to use Ansible to manage AWS cloud resources, covering the prerequisites for authentication and demonstrating how to create and launch resources like key pairs and EC2 instances.

17.1 Prerequisites: Authentication and Environment Setup

To manage AWS services, Ansible needs to authenticate with your AWS account. This is typically done by setting environment variables.

- **IAM User:** Create an IAM user in your AWS account with appropriate permissions (e.g., AdministratorAccess for full control).
- **Access Keys:** Generate an access key and secret key for this user. These are highly sensitive and should be handled with care.
- **Environment Variables:** Export the access keys and the desired AWS region as environment variables.
 - `export AWS_ACCESS_KEY_ID=<your-access-key>`
 - `export AWS_SECRET_ACCESS_KEY=<your-secret-key>`
 - `export AWS_REGION=<your-region-code>` (e.g., us-east-1)
- **Persistent Variables:** To avoid re-exporting these variables every time, you can add them to your shell's profile file (e.g., `~/.bashrc` or `~/.zshrc`) and then run `source ~/.bashrc`.
- **Python Libraries:** Ansible modules for AWS depend on the Python Boto3 library. You must install it using `pip` on your control machine.
 - `pip install boto3`

17.2 The Ansible Playbook Structure

Playbooks for AWS automation follow a specific structure:

- **hosts: localhost:** All AWS management tasks are run on the Ansible control machine, which makes API calls to AWS.
- **gather_facts: false:** This is recommended because you don't need to gather facts about the control machine itself for these tasks.
- **tasks:** A list of tasks using specific AWS modules.

17.3 Key AWS Modules and Examples

17.3.1 Creating an EC2 Key Pair

The **amazon.aws.ec2_key** module is used to create a new key pair. You can store the private key that is generated as a local file for later use.

- **Registering Output:** Use register: <variable_name> to save the output of the task into a variable. This is essential for accessing the generated private key.
- **Saving the Private Key:** Use the ansible.builtin.copy module with a when condition to save the private key to a local file. The when condition ensures the copy task only runs if the key was actually created.
 - when: key_out.changed

Playbook Example:

YAML

- name: Create EC2 key pair

amazon.aws.ec2_key:

name: sample

region: us-east-1

register: key_out

- name: Save key to file

ansible.builtin.copy:

content: "{{ key_out.key.private_key }}"

dest: "./sample.pem"

mode: '0600'

when: key_out.changed

why this condition? Because when the key created first time, then the changed:true condition would be there and we can save the key. Otherwise, if the key already there then, we cannot save the key. See below ss to understand this.

```
PLAY [localhost] *****

TASK [Create a keypair] *****
changed: [localhost]

TASK [print the private key] *****
ok: [localhost] => {
  "aws_private_key": {
    "changed": true,
    "failed": false,
    "key": {
      "fingerprint": "2c:e3:8d:b3:63:25:ed:04:49:7f:82:80:da:38:ab:09:d6:38:e9:d1",
      "id": "key-07677c3ce337fcb8a",
      "name": "my_keypair",
      "private_key": "-----BEGIN RSA PRIVATE KEY-----\nMIIEpAIBAAKCAQEAznZOInSkQIOMBHmD+lvIg+bAHaK09oaU
yyAKCVFQm1BCS\niwmjM/77R9e4Lyp1e9vPxoFLDPk1Qj30ys+jqcuIkVafq18nQr6+8m/u8/SET1rb+\nyNaU99Yi7onJIEhxJ6Q+EBTi2YR
TpTnVbP1SDz3nwk4o41zmb9p10xw9EMEKV\nnnbL7LfBeUWtsdMa1AxuaAnHig3VY54UIfZFqbFDKSe5ivNotUEE7AgNKTfanCxMS\nnpN1E1x
a5EozMqagrJyd7h2PBZpcvUf+yJTyglu5zbIkH3aEucje6vIHADJZUdv\nn8KztPvH15RCYr2DtD/RJnKAA3WP3+wc9GC914QIDAQABAoIBAGQ
zex2JpxSuJh\nnF+2nXmqmV8GOIEaGiWomGz6ysr/7F75FTNquktD1jHIQVyi/uJejTmOLr7dNTwxV\nd4ZaRW/80iXAmfQhuOeMI76RL+i rcd
fSR9AVB1xiVoM/w3MzRKQeULtG7Ezpf4\nn/zwfXj9q/vG8zxvFwNCO5T2H0J2jslwOACRtA4k9VCngmIMYkwcZvvKB51QQX72y\nnp2PBjLwOY
L2mEfsJZYx/wb7+zDBgXP3aAdNFwopv43wvb417BsrBek4AiBxOd6\nn2myzTY6ysGRYADDz5tVeArbw1Tfvv95Zzuz++AAKMi i INjoqZWYuuu
FkJTfpOX\nnmGOTMAECgYEA++H9DhbqhyeoZBL9RusYpbs1vaLH2cWJFIB2aYTW1rgFYXRmj\nnTYL1Ti3trf//8MZ5Uj7fOXUGVESXK8+Fpr
m5ZbKbPv8Hua1ihfD4BJv5k8b4cI6\nnPzswkh4zVytqfFFpS98yD2v/7en8Is1kqw3fLkpLMje18/sQpmxCfMkCgYEA0dZA\nnqq1yHimfd0nn
Oe/OMgzCOWny35d4L1DqZQOk2RHuIN8EjKIKYd9CJ4G3sta6I\nnd+OxfQpprBZZ1cUawrvHMZgn7K1r0z/XGfxNv7qoaUEa/jf0FufzGTVOH
6jVm3\nd3Yh+SG8gqwexbPH0BaGKRJmmurxhHOaOf7PdFkCgYBkw/Gic1R6+tw4YYfh1KKQ\nn8mS7EZYyh+D54noYuk9vLOEKbX98j0cfOSBW
j8Of+6Qab97qQnseeySNrEwBf\nnCzaF3p8PpzyeGQY6/NFbtrJODIjznPUE+T5BSow2bON7Fu9eJrkRIZSPVEa8yQ0\nns1X4tMwEWPLQ+j2
m5a0QK8gQCyaS130ipkdkSg2pajfeRZagFQjAWAU3HHutgd\nn1pxewbmk3RtRGTknUi rgMfRVhutzUw4KNx1t12DQEWICnK54IK67v8joRU3o
bw\nnf5hpXSU1G4s3sOYmbjdpgkoim/YlZb/jwz7M55BRSoln9NKST2FbFwibb+ZeqyS\nnmJ0LeQK8gQCw8fMyOfTSrnvjH+ya/Rfeyt1qCaa
ZQMktgzNjk2SFHsfh4LZH0Q\ngt6vpK9EKVCzEo2UP+gpM6qyhm7opPOMOSgbt10Gwozu1CAj7DsYtZKcmu8I0M\nn/h25JqEULu16ht8I3
1fvm13+dd766NqKc9ubrrwki7p8YKjw1A--\n-----END RSA PRIVATE KEY-----"
```

When `changed:false` the key value is not there, hence we cannot save the key in local

```
PLAY [localhost] *****

TASK [Create a keypair] *****
ok: [localhost]

TASK [print the private key] *****
ok: [localhost] => {
  "aws_private_key": {
    "changed": false,
    "failed": false,
    "key": {
      "fingerprint": "97:de:83:89:d1:09:ac:7a:da:59:45:eb:29:93:ed:0e:b9:99:80:67",
      "id": "key-0d013a4910aace2c0",
      "name": "my_keypair",
      "tags": {},
      "type": "rsa"
    },
    "msg": "key pair already exists"
  }
}

PLAY RECAP *****
localhost : ok=2    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

17.3.2 Launching an EC2 Instance

The `amazon.aws.ec2_instance` module launches new EC2 instances.

- **Image ID (`image_id`):** Specify the Amazon Machine Image (AMI) ID. This is region-specific, so ensure you get the correct ID for your chosen region.
- **Instance Type (`instance_type`):** Specify the size of the instance (e.g., `t2.micro`).
- **Key Name (`key_name`):** Use the name of the key pair you created earlier to enable SSH access.
- **Region (`region`):** Specify the region where you want to launch the instance.
- **`exact_count: 1`:** This parameter is crucial. It tells Ansible to ensure there is only one instance matching the specified parameters. If you run the playbook again and an instance already exists, it will not launch a new one. Without this, repeated runs could create multiple instances.

Playbook Example:

YAML

- name: Launch an EC2 instance

amazon.aws.ec2_instance:

key_name: sample

instance_type: t2.micro

image_id: ami-0c55b159cbfafa1f0 # Example AMI for US East (N. Virginia)

region: us-east-1

exact_count: 1

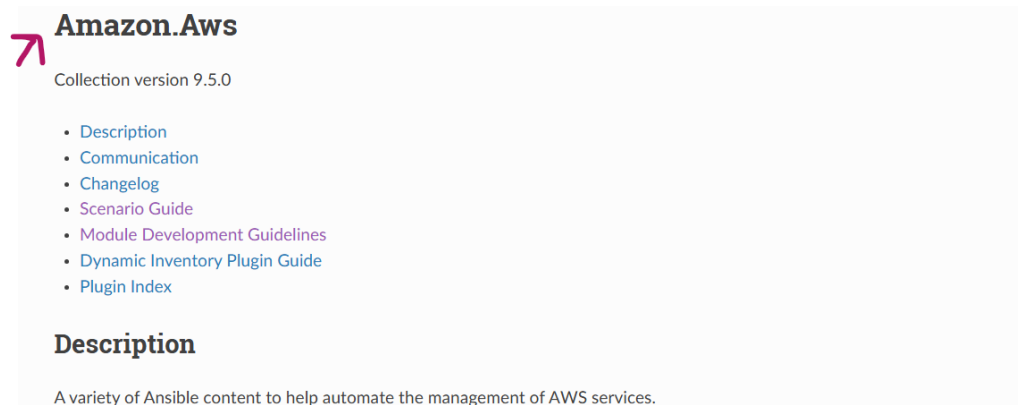
wait: true

state: running

17.4 Key Takeaways

- Ansible's AWS modules work by making **API calls** to your cloud account from the control machine.
- **Proper authentication** required Python libraries (boto3) are essential prerequisites.
- Always use exact_count when launching instances to prevent duplicates on repeated playbook runs.
- **Ansible documentation** is the best resource for finding specific modules and their required parameters. Documentation link - <https://docs.ansible.com/ansible/latest/collections/amazon/aws/index.html>

So, collection name is amazon.aws



Amazon.Aws
Collection version 9.5.0

- [Description](#)
- [Communication](#)
- [Changelog](#)
- [Scenario Guide](#)
- [Module Development Guidelines](#)
- [Dynamic Inventory Plugin Guide](#)
- [Plugin Index](#)

Description

A variety of Ansible content to help automate the management of AWS services.

And these are the modules

Modules

- [autoscaling_group module](#) – Create or delete AWS AutoScaling Groups (ASGs)
- [autoscaling_group_info module](#) – Gather information about EC2 Auto Scaling Groups (ASGs) in AWS
- [autoscaling_instance module](#) – manage instances associated with AWS AutoScaling Groups (ASGs)
- [autoscaling_instance_info module](#) – describe instances associated with AWS AutoScaling Groups (ASGs)
- [autoscaling_instance_refresh module](#) – Start or cancel an EC2 Auto Scaling Group (ASG) instance refresh in AWS
- [autoscaling_instance_refresh_info module](#) – Gather information about EC2 Auto Scaling Group (ASG) Instance Refreshes in AWS
- [aws_az_info module](#) – Gather information about availability zones in AWS
- [aws_caller_info module](#) – Get information about the user and account being used to make AWS calls
- [aws_region_info module](#) – Gather information about AWS regions
- [backup_plan module](#) – Manage AWS Backup Plans
- [ec2_eip_info module](#) – List EC2 EIP details
- [ec2_eni module](#) – Create and optionally attach an Elastic Network Interface (ENI) to an instance
- [ec2_eni_info module](#) – Gather information about EC2 ENI interfaces in AWS
- [ec2_import_image module](#) – Manage AWS EC2 import image tasks
- [ec2_import_image_info module](#) – Gather information about import virtual machine tasks
- [ec2_instance module](#) – Create & manage EC2 instances
- [ec2_instance_info module](#) – Gather information about ec2 instances in AWS
- [ec2_key module](#) – Create or delete an EC2 key pair
- [ec2_key_info module](#) – Gather information about EC2 key pairs in AWS
- [ec2_launch_template module](#) – Manage EC2 launch templates
- [ec2_launch_template_info module](#) – Gather information about launch templates and versions
- [ec2_metadata_facts module](#) – Gathers facts (instance metadata) about remote hosts within EC2

we have used modules `ec2_instance` and `ec2_key`. So in playbook the module names that we use
→ `amazon.aws.ec2_instance` and `amazon.aws.ec2_key`

- While Ansible is a powerful tool for cloud provisioning, more specialized tools like **Terraform** often provide more comprehensive solutions for infrastructure management.

